

DIABETES PREDICTION SYSTEM USING A LEAKAGE-SAFE ARTIFICIAL NEURAL NETWORK

A PROJECT REPORT

submitted for the course

21CSE326T – ARTIFICIAL NEURAL NETWORKS

Submitted by

SHANMUGAPRIYAN J (RA2411026050011)
DEEPAK V F (RA2411026050015)

III Year / V Semester, Section A
B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning

SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI – 621105

OCTOBER 2026

ABSTRACT

According to the World Health Organization, at present, about 830 million people in the world are living with diabetes mellitus. The scope of this course is narrower: can a handful of easily affordable clinical measures point to which patients are most likely to test positive? The Pima Indians diabetes dataset is the conventional working standard for that task. It was compiled by the National Institute of Diabetes and Digestive and Kidney Diseases from adult Pima women near Phoenix, Arizona. In 1988, Smith et al. had already trained an initial neural model on the collection. UCI still stores it with a total of 768 records. The variables included in the record indicate the number of pregnancies, 2-hour plasma glucose, diastolic blood pressure, triceps skinfold thickness, 2-hour serum insulin, BMI, pedigree score, and age.

The published accuracies in the present document fluctuate considerably. There has been information from studies whose recorded accuracy is approximately 76% and others that exceeded 90%. We found that most of those papers were based on the same protocol. The preprocessing was applied to all of the 768 entries before any train–test separation. Zeros in glucose or insulin were counted as real results, even though these values would be impossible, given the physiology of an organism. Although accuracy is often treated as the leading criteria, a sizeable portion (65.1%) of the entries contains negative cases.

In this work, a small feed-forward artificial neural network has been trained while avoiding the use of shortcuts. Impossible zeros for glucose, blood pressure, thickness of skin, insulin levels and BMI have been changed into missing values; a tally of zero pregnancies remains unchanged. The median imputation and standardisation have been estimated only from the training fold. Oversampling of the minority class has been carried out in the training fold, and then an 8–16–8–1 neural network with 289 weights has been trained using Adam, small L2 penalty and early stopping.

In the same folds, logistic regression, random forest and dummy classifier were evaluated. The values of accuracy, precision, sensitivity, specificity, F1, ROC-AUC, PR-AUC and confusion matrices are reported. In 10 x 5 repeated cross-validation, the network achieved sensitivity of 0.768, specificity of 0.734, F1 of 0.679, and ROC-AUC of 0.834. In the holdout test comprising 154 records 43 out of 54 diabetes patients were identified correctly (ROC-AUC of 0.821). Logistic regression (ROC-AUC of 0.836) and random forest (ROC-AUC of 0.838) scored marginally higher than the developed network. Oversampling in the training fold increased sensitivity by 0.208 ($p < 0.001$). The elimination of pipeline elements resulted in a fall of ROC-AUC to 0.721 owing to the absence of standardisation. In the same manner, the elimination of early stopping led to a reduction of the ROC-AUC to 0.784, and the removal of medical zeros ruled ROC-AUC dropped the value to 0.820. Achieveing deeper models turned out to have a minor influence, as well as activation choice or L2 regularisation applied, as well as univariate feature selection.

The oversampling performed before the cross-validation raised the value of PR-AUC from 0.706 to 0.825. In contrast to that finding, the nested cross-validation showed drop of the best tuned ROC-AUC from 0.835 to 0.819.

Ten different random seeds resulted in a ROC-AUC fluctuation value around 0.04, which is in line with most architectural changes.

The foremost feature that appears to have a significant impact on accuracy is glucose, being followed by age, BMI and number of pregnancies.
Keywords: artificial neural network, multilayer perceptron, diabetes prediction, Pima Indians Diabetes dataset, data leakage, class imbalance, ablation study.

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Background 1
1.2 The Pima Indians Diabetes Benchmark 1
1.3 Problem Statement 1
1.4 Motivation 2
1.5 Objectives 2
1.6 Scope and Limitations 2
1.7 Contributions 2
1.8 Organisation of the Report 3
2. LITERATURE SURVEY 4
2.1 Foundational Study 4
2.2 Neural-Network Approaches on the Pima Dataset 4
2.3 Classical and Ensemble Machine-Learning Approaches 4
2.4 Methodological Literature 5
2.5 Comparison of Existing Work 5
2.6 Research Gap 6
3. PROPOSED METHODOLOGY 8
3.1 System Overview and Block Schematic 8
3.2 Problem Formulation 8
3.3 Data Preprocessing 9
3.4 Artificial Neural Network Architecture 9
3.5 Baseline Models 11
3.6 Evaluation Protocol and Metrics 11
3.7 Algorithms 13
3.8 Implementation Details 14
4. EXPERIMENTAL RESULTS AND DISCUSSIONS 15
4.1 Experimental Setup 15
4.2 Dataset Details 15
4.3 Exploratory Data Analysis 17
4.4 Feature Relevance and Selection 19
4.5 Training Behaviour of the ANN 20
4.6 Cross-Validated Comparison of All Configurations 21
4.7 Repeated Cross-Validation and Statistical Significance 22
4.8 ROC, Precision–Recall, and Confusion Matrices 24
4.9 Decision-Threshold Analysis 25
4.10 Hold-Out Test Evaluation 25
4.11 Interpretability: Permutation Importance 27
4.12 System Output Demonstration 28
4.13 Discussion 29

5. ABLATION STUDY AND STATE-OF-THE-ART ANALYSIS 30
5.1 Ablation Protocol.....30
5.2 Component Ablation.....30
5.3 Architecture and Hyperparameter Ablation.....32
5.4 Leakage Ablation.....34
5.5 Nested versus Non-Nested Hyperparameter Selection.....36
5.6 Seed Stability.....37
5.7 State-of-the-Art Comparison.....37
5.8 Summary of Findings.....39
6. CONCLUSION AND FUTURE WORK 40
6.1 Conclusion.....40
6.2 Future Work.....40
REFERENCES.....42

LIST OF FIGURES

Fig 3.1: Block schematic of the proposed system 8
Fig 3.2: Architecture of the proposed ANN (8–16–8–1).....10
Fig 3.3: Stratified 5-fold cross-validation protocol 12
Fig 4.1: Outcome class balance 16
Fig 4.2: Zero values per feature.....17
Fig 4.3: Feature distributions by outcome 17
Fig 4.4: Box plots by outcome 18
Fig 4.5: Correlation matrix 19
Fig 4.6: ANOVA F-scores and selection frequency.....19
Fig 4.7: Training loss and validation accuracy 20
Fig 4.8: Learning curve of the proposed ANN 20
Fig 4.9: Console output: seeded 5-fold comparison 22
Fig 4.10: Repeated cross-validation scores 23
Fig 4.11: Console output: repeated CV and t-tests 24
Fig 4.12: Out-of-fold ROC and precision–recall curves 24
Fig 4.13: Out-of-fold confusion matrices 25
Fig 4.14: Decision-threshold analysis 25
Fig 4.15: Hold-out ROC curves and confusion matrix.....26
Fig 4.16: Console output: hold-out evaluation 27
Fig 4.17: Permutation importance 28
Fig 4.18: Output of the prediction module 28
Fig 4.19: Unit-test output 29
Fig 5.1: Component ablation chart 31
Fig 5.2: Console output: component ablation.....31
Fig 5.3: Architecture and hyperparameter ablation chart 34

Fig 5.4: Leakage ablation chart 35
Fig 5.5: Console output: leakage and nested CV 36
Fig 5.6: Seed stability of the proposed ANN 37

LIST OF TABLES

Table 2.1: Comparison of published work on the Pima dataset 6
Table 3.1: Hyperparameters and settings 14
Table 4.1: Dataset summary 15
Table 4.2: Feature dictionary and statistics 16
Table 4.3: Seeded 5-fold comparison of all configurations 21
Table 4.4: Repeated cross-validation of the main models 22
Table 4.5: Corrected resampled t-tests 23
Table 4.6: Hold-out test results 26
Table 5.1: Component ablation 30
Table 5.2: Architecture and hyperparameter ablation 33
Table 5.3: Leakage ablation 35
Table 5.4: Nested cross-validation results 36
Table 5.5: State-of-the-art comparison 38

1. INTRODUCTION

1.1 Background

Diabetes mellitus is not one disease. The body either makes too little insulin or cannot use the insulin it has. Glucose stays high. Over years that eats into vessels, kidneys, eyes and nerves. WHO lists it among the main drivers of heart attack, stroke, kidney failure, blindness and amputation [1]. Adult cases went from about 200 million in 1990 to about 830 million in 2022. Prevalence roughly doubled, 7% to 14% [1], [2]. IDF thinks the curve keeps climbing through 2045 [3]. Plenty of type 2 cases are still missed. A cheap flag from routine numbers will not diagnose anyone. It can still decide who gets tested first.

Clinic diagnosis still uses lab cut-offs. Fasting plasma glucose ≥ 126 mg/dL. Two-hour OGTT plasma glucose ≥ 200 mg/dL. Or HbA1c ≥ 6.5% [4]. We are not replacing those tests. The job here is risk ranking: take a handful of measurements and guess how likely a person is to come back positive, so the scarce tests go where they might matter. Machine learning already sits in a lot of diabetes papers [5]. ANNs show up in a big share of them.

A feed-forward ANN (multilayer perceptron) pushes an input through weighted sums and nonlinear activations and ends with a probability. Give it enough hidden units and it can approximate any continuous function on a bounded set [6]. Weights come from gradient descent plus back-propagation [7]. That is handy when glucose, BMI and age may interact. It is also how you over-fit a 768-row table if you are not careful. That tension is the actual project.

1.2 The Pima Indians Diabetes Benchmark

Gila River, near Phoenix, has one of the highest recorded type 2 rates anywhere [8]. Knowler et al. put the incidence at about 19 times Rochester, Minnesota [9]. NIDDK kept examining the community from 1965. In 1988 Smith et al. pulled 768 exams of women aged 21 or older and tried to forecast diabetes inside five years with ADAP, an early neural routine [10]. UCI got the extract in 1990 [11]. Coursework has used it ever since.

The file is small, all numbers, binary label. Easy for a laptop project. It also has traps. Classes are off: 500 negatives (65.1%) and 268 positives (34.9%). Always say "no diabetes" and you already hit 65.1% accuracy. Older notes claim no missing values. Then you look at glucose, blood pressure, skinfold, insulin and BMI. Zeros. A living adult cannot have those. They are missing-value codes [12]–[14]. With only 768 rows, a little leak from the test side into training is enough to puff the scores up.

1.3 Problem Statement

Given eight numbers for an adult Pima woman (pregnancies, 2-hour OGTT glucose, diastolic blood pressure, triceps skinfold, 2-hour serum insulin, BMI, pedigree, age), estimate $P(y = 1)$ under the study rule and then cut that probability into a class. The model has to be an ANN. Nothing computed from an evaluation row can leak into fitting. We compare it with simpler baselines on the same splits. Metrics have to still mean something when 65% of rows are negative.

1.4 Motivation

Leakage is the boring reason a lot of applied papers look stronger than they are: evaluation data sneaks into model building [15], [16]. On Pima it is usually the imputer, scaler or selector fitted on all 768 rows before CV. Or oversampling before the split, so copies of the same woman sit in train and test. Or tuning on the same folds you later quote. Reporting is the other mess. Bozkurt et al. already said many older Pima ANN papers quoted accuracy only [17], which is a weak summary when classes are skewed [18]. Prediction-model checklists also want a clear missing-data story and a real validation [19]. We wanted the lab task done tightly enough that the numbers hold, and we wanted a number on how much the usual shortcuts actually move them.

1.5 Objectives

1. Load the dataset from a local file only, profile class balance, duplicates, missing values, and zero codes, and write down every data decision.
2. Recode medically impossible zeros as missing in five measurements, while keeping Pregnancies = 0 as a valid value.
3. Build a pipeline in which imputation, optional feature selection, scaling, and oversampling are fitted on the training fold only.
4. Design and train a feed-forward ANN (8–16–8–1) with ReLU hidden units, a sigmoid output, Adam, L2 regularisation, and early stopping.
5. Compare the ANN with a dummy classifier, logistic regression, and a random forest on the same stratified folds, and test differences with a corrected resampled t-test.
6. Report accuracy, precision, sensitivity, specificity, F1, ROC-AUC, PR-AUC, and confusion matrices, and score the model once on an untouched hold-out set.
7. Run an ablation of pipeline pieces and ANN hyperparameters, and measure the effect of leakage and of non-nested hyperparameter selection.
8. Compare the results with published Pima studies and explain where the protocols differ.

1.6 Scope and Limitations

Binary classification on 768 Pima rows. Small MLP plus classical baselines. Laptop CPU. Not a clinical study. The population is narrow: adult women from one community, 1980s exams, a historical WHO-style glucose rule, nearly half the insulin values missing, no outside cohort. Do not use these numbers for diagnosis. Do not push them onto men, children or other groups.

1.7 Contributions

- A reproducible, leakage-safe ANN pipeline for the Pima benchmark, with a medically justified zero policy, implemented as a tested Python package (20 unit tests).
- A fair comparison of the ANN with three baselines on the same folds, using repeated cross-validation, corrected significance tests, bootstrap intervals, and a single hold-out evaluation.

- An ablation of more than thirty configurations that shows which design choices actually matter for the ANN.
- A controlled leakage experiment that measures how much common protocol shortcuts inflate F1, ROC-AUC, and PR-AUC, which helps explain why published numbers sit so far apart.

1.8 Organisation of the Report

Chapter 2 is the literature: Pima ANN papers, classical models, and the methods work on leakage and imbalance. It ends with a comparison table and the gap we actually chased. Chapter 3 is the method (block diagram, equations, algorithms). Chapter 4 is the data and the numbers, with console screenshots. Chapter 5 is ablation plus published scores. Chapter 6 is the wrap-up and what we would do next.

2. LITERATURE SURVEY

Four piles of papers. The original Pima modelling study. Later neural nets on the same table. Classical and ensemble methods. Then the methods papers on evaluation. We quote the numbers as the authors wrote them. Protocols are not the same from paper to paper. Section 2.6 and Chapter 5 pick that apart.

2.1 Foundational Study

Smith et al. [10] are the reason this file exists as a modelling task. They used ADAP, an adaptive learning routine built from perceptron-like devices, to guess who would develop non-insulin-dependent diabetes inside five years. The label was not a clinic diagnosis in the modern sense. It was a 2-hour post-load plasma glucose of at least 200 mg/dL, either at a survey exam or in routine care. They trained on 576 of the 768 eligible exams and held out 192. Cut the real-valued output at 0.448 and both sensitivity and specificity landed at 76%. They also drew a full ROC curve. That last bit still looks unusually honest. A lot of later papers just quoted accuracy and stopped.

2.2 Neural-Network Approaches on the Pima Dataset

Kayaer and Yıldırım [20] tried multilayer perceptrons, radial basis function networks, and a general regression neural network (GRNN) on the UCI extract. Ordinary train/test split. The GRNN came out on top at 80.21% test accuracy. An MLP trained with Levenberg–Marquardt sat at 77.08%. Not a huge gap. Polat and Güneş [21] squeezed the eight features down to four principal components, classified them with ANFIS, and quoted 89.47%. Kahramanli and Allahverdi [22] mixed a crisp ANN with a fuzzy net trained by back-propagation. Their k-fold figure was 84.24%. Different recipes, different headlines. Lining them up is already a bit of a stretch.

Temurtas et al. [23] put an LM-trained multilayer net next to a probabilistic neural network, with both a split and 10-fold CV. The multilayer net hit 82.37%. The PNN hit 78.13%. Later tables copy those numbers [24]. Erkaymaz et al. [24] rewired a feed-forward net into small-world graphs. Newman–Watts: 93.06% accuracy, sensitivity 0.85, specificity 0.96. Watts–Strogatz: 91.66%. Ordinary feed-forward: 83.33%. The graphs get fancier. Accuracy is still the number most papers lead with.

2.3 Classical and Ensemble Machine-Learning Approaches

Classical models sit in a narrower band. Maniruzzaman et al. [25] ran Gaussian-process classification against LDA, QDA and naïve Bayes under several CV setups. The Gaussian-process model quoted 81.97% accuracy, 91.79% sensitivity, 63.33% specificity. High recall, weak specificity. Sisodia and Sisodia [26] trained a tree, an SVM and naïve Bayes and actually reported precision, recall, F-measure and ROC. Naïve Bayes won on accuracy at 76.30%. Zou et al. [27] tried trees, forests and nets with PCA and mRMR, both on a Luzhou hospital set and on Pima. Joshi and Dhakal [28] kept it simpler: logistic regression and a decision tree. Logistic hit 78.26% and the useful predictors were pregnancies, glucose, BMI, pedigree and age. Chang et al. [29] compared naïve Bayes, a random forest and J48 on a 70/30 split. Forest: 79.57% accuracy, AUC 86.24% on the full feature set.

The headline numbers jump once people stack ensembles and heavy preprocessing. Hasan et al. [30] piled on outlier rejection, filling, standardisation, feature selection and k-fold CV, then several classifiers including an MLP. Weighted ensemble: AUC 0.950, sensitivity 0.789, specificity 0.934. Kumari et al. [31] stayed milder. Soft vote of random forest, logistic regression and naïve Bayes: 79.04%. Howlader et al. {c('howlader2022')} tried a lot of selectors and classifiers and quoted 90.91% for generalised boosted regression. That is well above the usual 75–82% band. Kavakiotis et al. [5] survey the wider diabetes ML literature if you want the bigger picture.

2.4 Methodological Literature

A few methods papers are the actual bar we tried to meet. Kohavi argued for stratified 10-fold CV when you pick a model and when you quote its accuracy [32]. Varma and Simon [33] and Cawley and Talbot [34] showed the trap: quote the CV score of a setup that was itself chosen by CV, and the number is biased high. Nested CV takes that bump out. Kaufman et al. [15] spelled out what counts as leakage. Kapoor and Narayanan [16] collected cases where it wrecked reproducibility. scikit-learn's own docs say every data-dependent step should sit inside a pipeline that is refitted in each fold [35]. We treated that as a rule, not a suggestion.

Imbalance is its own mess. He and Garcia [36] cover resampling and cost-sensitive tricks. Chawla et al. [37] introduced SMOTE, which interpolates extra minority rows. Fine, as long as it stays on the training side. Copies or interpolations of a test patient leaking into training is the usual failure. Fawcett [38] is still the ROC walkthrough we used. Davis and Goadrich [39] relate ROC to precision–recall. Saito and Rehmsmeier [18] make the point that PR plots say more than ROC when positives are the minority. Little and Rubin [40] say when a simple imputer is acceptable. That stack is the evaluation standard we held ourselves to.

2.5 Comparison of Existing Work

Table 2.1 lines the surveyed papers up by method, evaluation protocol, reported result, and the limitation that matters for this project. "Accuracy" here means the figure the authors quoted on their own protocol.

Table 2.1: Comparison of published work on the Pima Indians Diabetes dataset

Ref.	Technique	Evaluation	Reported result	Limitation / observation
[10]	ADAP (early neural model)	576/192 split	Sens. = Spec. = 76%; ROC	Single split; early algorithm
[20]	GRNN, RBF, MLP-LM	Train/test split	GRNN 80.21%; MLP-LM 77.08%	Accuracy only; single split
[21]	PCA (8→4) + ANFIS	10-fold CV	Accuracy 89.47%	Accuracy only
[22]	Hybrid ANN + fuzzy NN	k-fold CV	Accuracy 84.24%	Complex hybrid; no classical baseline
[23]	MLNN-LM, PNN	Split and 10-fold CV	MLNN 82.37%; PNN 78.13%	Accuracy only
[17]	Comparison of earlier classifiers	Literature + experiments	Most earlier studies report accuracy only	Highlights missing sensitivity/specificity
[24]	Small-world feed-forward ANN	k-fold CV	93.06%; Sens. 0.85, Spec. 0.96	Specialised topology; not independently replicated
[25]	Gaussian process, LDA, QDA, NB	K2–K10 CV	GPC 81.97%; Sens. 91.79%; Spec. 63.33%	Low specificity at reported operating point
[26]	DT, SVM, naïve Bayes	Cross-validation	NB accuracy 76.30%	No neural network compared
[30]	Outlier removal, selection, MLP & ensembles	k-fold CV	AUC 0.950; Sens. 0.789; Spec. 0.934	Outlier rejection changes the evaluated population
[28]	Logistic regression, decision tree	Train/test + CV	Accuracy 78.26%	No neural network compared
[31]	Soft-voting RF + LR + NB	Train/test	Accuracy 79.04%	No neural network in ensemble
[41]	Many selectors and classifiers	Cross-validation	GBR accuracy 90.91%	Well above the typical 75–82% range
[29]	NB, RF, J48	70/30 split	RF 79.57%; AUC 86.24%	Single split
This work	**Leakage-safe MLP 8–16–8–1 + training-fold ROS**	**10×5 repeated CV + hold-out**	**ROC-AUC 0.834; Sens. 0.768; Spec. 0.734**	**Single population; no external cohort**

2.6 Research Gap

Nobody needed another reminder that an ANN can be trained on Pima. Smith et al. did that in 1988. Dozens of papers followed. The hole is in how the work is done. You can see it in Table 2.1:

- Papers treat the zero codes differently — keeping them, recoding every zero including valid zero pregnancies, or dropping rows — and often do not say which rule they used.

- If preprocessing, feature selection, oversampling, or hyperparameter search is fitted on the full table or tuned on test data, the quoted scores look better than they are. Many write-ups do not make the protocol clear.
- Accuracy is still the headline number in most studies, even though the majority-class floor is 65.1%. None of the surveyed papers reports PR-AUC.
- ANNs are often shown without same-fold baselines, repeated cross-validation, or a significance test, so it is hard to tell whether the network actually beats logistic regression.
- How much each protocol shortcut inflates the scores is rarely measured on this dataset.

We tried to close that gap with a leakage-safe ANN pipeline, same-fold baselines plus corrected tests, a full metric set, an ablation, and a leakage run that actually measures how much the usual shortcuts puff the scores.

3. PROPOSED METHODOLOGY

3.1 System Overview and Block Schematic

Figure 3.1 is the whole pipeline on one page. Stage A just loads the local file, looks at it, and cuts it into stratified folds. Stage B is the training side. For each training fold we recode the impossible zeros, learn the medians, maybe pick features, learn the scaler, oversample the minority class, and fit the ANN. Stage C is the test fold. It goes through those already-fitted objects. No refitting. No extra resampling. Then the trained net, the 0.5 cut, and the scores. Stage D is bookkeeping: fold scores, baseline comparisons, ablation, leakage runs, hold-out.

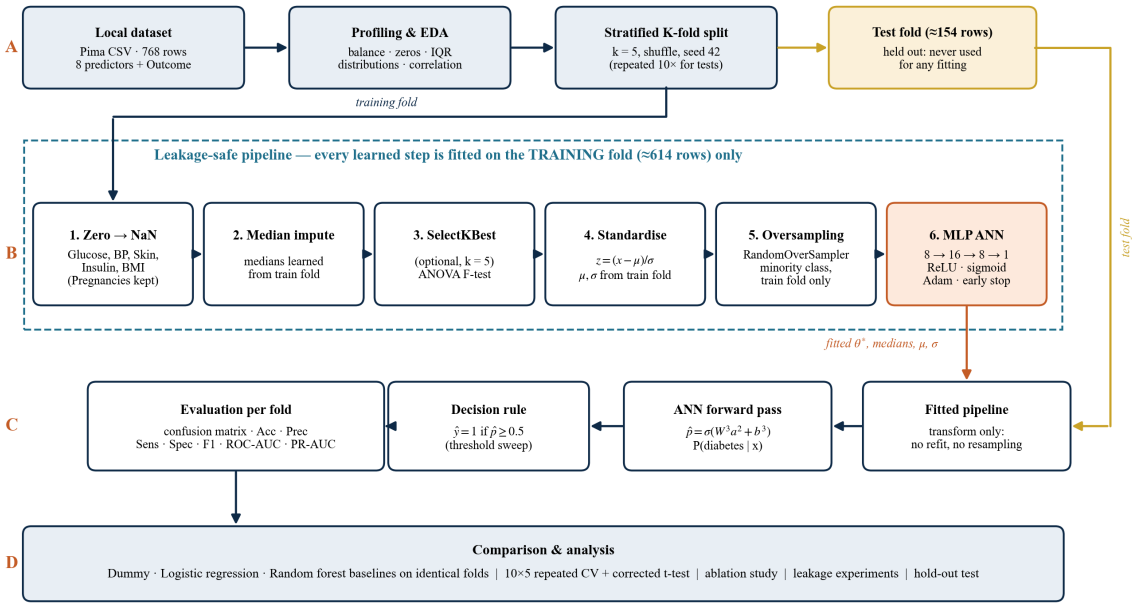


Figure 3.1: Block schematic of the proposed leakage-safe ANN-based Diabetes Prediction System. Learned steps (Stage B) are fitted on the training fold only; the test fold (Stage C) is transformed with the fitted objects and scored once.

3.2 Problem Formulation

Write D = $\{(x_i, y_i)\}$, $i = 1, \ldots, N$ with $N = 768$. Each x_i is the eight measurements. Each y_i is 0 or 1. We learn a function f_θ that estimates the chance of a positive label. A threshold τ then turns that chance into a class:

$$\hat{p}_i = f_\theta(x_i) \approx P(y_i = 1 \mid x_i), \qquad f_\theta : \mathbb{R}^8 \to [0, 1] \tag{3.1}$$

$$\hat{y}_i = \mathbf{1}[\hat{p}_i \geq \tau], \qquad \tau = 0.5 \tag{3.2}$$

Scores are always computed on rows that were not used to fit any part of the pipeline. If the indices are split into a training part T and a test part S, then medians, means, variances, F-scores, resampled rows, and network weights are all functions of the rows in T only.

3.3 Data Preprocessing

Zero recoding. Five measurements cannot be zero in a living adult: plasma glucose, diastolic blood pressure, triceps skinfold, 2-hour serum insulin, and BMI. Those zeros are turned into missing values (NaN) by a fixed rule that does not use any statistics, so it can be applied the same way to every row. Zero pregnancies is a real value and is kept. With Z the set of recoded columns:

$$\tilde{x}_{ij} = \text{NaN} \text{ if } j \in Z \text{ and } x_{ij} = 0; \quad \tilde{x}_{ij} = x_{ij} \text{ otherwise}, \quad Z = \{\text{Glu}, \text{BP}, \text{Skin}, \text{Ins}, \text{BMI}\} \tag{3.3}$$

Median imputation. Each missing value in column j is filled with the median of the observed training-fold values of that column [40]. The median holds up against the long right tails of insulin and the pedigree function:

$$m_j = \text{median}\{\tilde{x}_{ij} : i \in T,\ \tilde{x}_{ij} \neq \text{NaN}\}, \qquad \hat{x}_{ij} = m_j \text{ if } \tilde{x}_{ij} = \text{NaN} \tag{3.4}$$

Univariate feature selection (optional). When it is switched on, SelectKBest keeps the k features with the largest one-way ANOVA F statistic on the training fold [42]. For feature j with class means $\bar{x}_{jc}$, overall mean $\bar{x}_j$, n_c rows in class c, and $C = 2$ classes:

$$F_j = \frac{\sum_c n_c (\bar{x}_{jc} - \bar{x}_j)^2 / (C-1)}{\sum_c \sum_{i \in c} (x_{ij} - \bar{x}_{jc})^2 / (n - C)} \tag{3.5}$$

Standardisation. Each feature is centred and scaled with the mean μ_j and standard deviation σ_j of the training fold. That puts glucose in mg/dL and the pedigree score on similar scales for gradient-based training:

$$z_{ij} = \frac{\hat{x}_{ij} - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{|T|} \sum_{i \in T} \hat{x}_{ij}, \quad \sigma_j^2 = \frac{1}{|T|} \sum_{i \in T} (\hat{x}_{ij} - \mu_j)^2 \tag{3.6}$$

Random oversampling of the training fold. To counter the 65:35 imbalance [36], minority-class rows of the training fold are copied at random (with replacement) until both classes have the same count. If I_1 is the index set of positive training rows and n_0, n_1 are the class counts, the resampled training set is:

$$T' = T \cup \{i_1, \ldots, i_{n_0 - n_1}\}, \qquad i_k \sim \text{Uniform}(I_1) \tag{3.7}$$

Oversampling is done with imbalanced-learn [43] inside the pipeline, so it runs only during fitting. The test fold is never resampled. That keeps its class balance the same as the population and stops copies of a test patient from entering training. SMOTE [37] is tried as an alternative in the ablation study.

3.4 Artificial Neural Network Architecture

The classifier is a fully connected feed-forward network with eight inputs, two hidden layers of 16 and 8 units, and one output (Figure 3.2). Hidden units use ReLU [44]; the output uses a logistic sigmoid so it can be read as a probability. The network has (8·16 + 16) + (16·8 + 8) + (8·1 + 1) = 289 trainable parameters. That is small next to the roughly 800 rows of an oversampled training fold, which helps keep over-fitting in check.

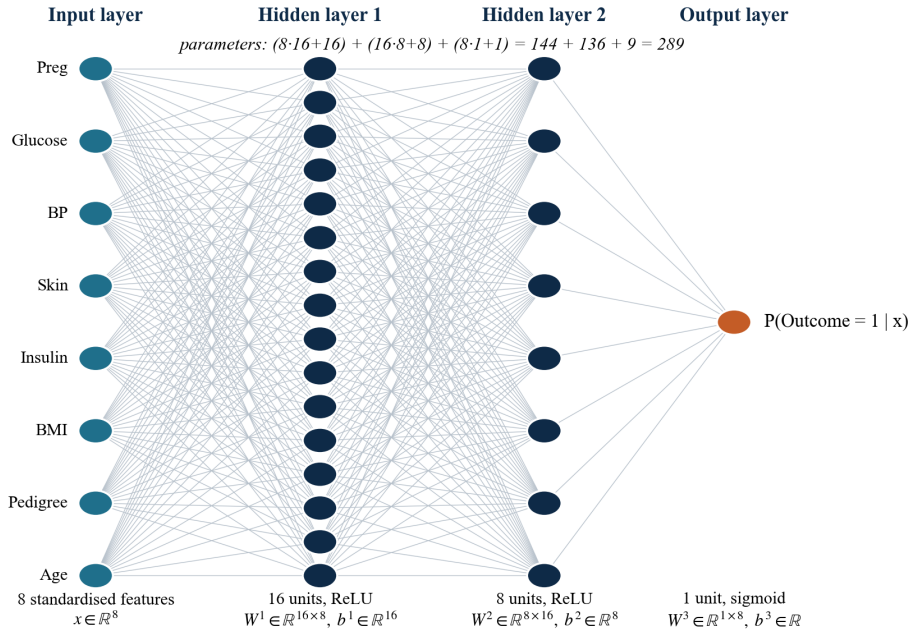


Figure 3.2: Architecture of the proposed feed-forward ANN (8–16–8–1) with ReLU hidden layers and a sigmoid output.

Forward propagation. With $a^{(0)} = z$ the standardised input, each hidden layer does an affine transform then ReLU, and the output layer applies the sigmoid:

$$h^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \mathrm{ReLU}(h^{(l)}) = \max(0, h^{(l)}), \qquad l = 1, 2 \tag{3.8}$$

$$\hat{p} = \sigma(W^{(3)} a^{(2)} + b^{(3)}), \qquad \sigma(u) = \frac{1}{1 + e^{-u}} \tag{3.9}$$

where $W^{(1)}$, $W^{(2)}$, and $W^{(3)}$ are matrices of size 16 × 8, 8 × 16, and 1 × 8 with matching bias vectors. Weights are initialised with the Glorot (Xavier) uniform scheme [45], where n_{in} and n_{out} are the fan-in and fan-out of the layer:

$$W_{rs}^{(l)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right) \tag{3.10}$$

Loss function. The network minimises mean binary cross-entropy over a mini-batch B, plus an L2 penalty on the weights with strength $\alpha = 10^{-3}$ (weight decay):

$$\mathcal{L}(\theta) = -\frac{1}{|B|} \sum_{i \in B} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \frac{\alpha}{2|B|} \sum_{l=1}^{3} \| W^{(l)} \|_F^2 \tag{3.11}$$

Back-propagation. Gradients come from the chain rule [7], [46]. For a sigmoid output with cross-entropy loss the output error is just the prediction minus the label. Errors are sent backwards through the ReLU derivative, and the weight gradient is the outer product of the error with the previous layer's activation:

$$\delta^{(3)} = \hat{p} - y, \qquad \delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \mathbf{1}[h^{(l)} > 0], \quad l = 2, 1 \tag{3.12}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{|B|} \sum_{i \in B} \delta_i^{(l)} a_i^{(l-1)\top} + \frac{\alpha}{|B|} W^{(l)}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{|B|} \sum_{i \in B} \delta_i^{(l)} \tag{3.13}$$

Adam optimiser. Parameters are updated with Adam [47] on mini-batches of 32 rows, with learning rate $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$. With gradient g_t at step t:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{3.14}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{3.15}$$

Early stopping. A stratified 15% of the (resampled) training fold is held out as a validation set [48]. Training stops when validation accuracy has not improved by at least 10^{-4} for 20 epochs in a row (maximum 500), and the weights from the best validation epoch are restored. Early stopping and the L2 penalty are the two regularisers of the network.

3.5 Baseline Models

Three baselines go through the same pipeline and the same folds. The **dummy classifier** always predicts the majority class and sets the floor (accuracy ≈ 0.651, sensitivity 0, ROC-AUC 0.5). **Logistic regression** [49] is a linear model of the log-odds with L2 regularisation (C = 1) and balanced class weights, solved with liblinear:

$$\hat{p} = \sigma(w^\top z + b), \qquad \omega_c = \frac{N}{2N_c} \ \text{(balanced class weight for class } c) \tag{3.16}$$

The **random forest** [50] averages 100 decision trees grown on bootstrap samples with a random subset of $\sqrt{8} \approx 3$ features considered at each split, maximum depth 6, at least 5 rows per leaf, and balanced class weights. Splits minimise Gini impurity, and the forest's probability is the mean of the trees' leaf class fractions:

$$G(t) = 1 - \sum_{c \in \{0,1\}} p_c(t)^2, \qquad \hat{p}_{RF}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b(x), \quad B = 100 \tag{3.17}$$

3.6 Evaluation Protocol and Metrics

Stratified k-fold cross-validation with $k = 5$ [32] splits the data into five folds that keep the same class ratio (Figure 3.3). Each model is trained five times, each time on four folds, and scored on the remaining fold. Every model sees the same splits. For significance testing the procedure is repeated ten times with different shuffles (50 splits). Separately, a single stratified 80/20 split gives a hold-out test set that is scored once after all design decisions were fixed.

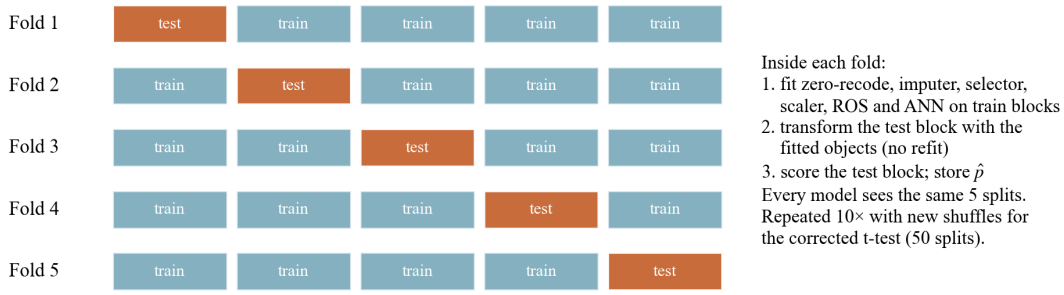


Figure 3.3: Stratified 5-fold cross-validation with the full pipeline refitted inside each training fold.

Write TP, TN, FP, and FN for the true positives, true negatives, false positives, and false negatives at threshold τ. The threshold-dependent metrics are then:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP} \tag{3.18}$$

$$\text{Sensitivity} = \text{Recall} = \frac{TP}{TP + FN}, \qquad \text{Specificity} = \frac{TN}{TN + FP} \tag{3.19}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.20}$$

Two threshold-free metrics summarise ranking quality. ROC-AUC [38], [51] is the chance that a randomly chosen positive is scored above a randomly chosen negative. PR-AUC is computed as average precision [18], [39]. It puts more weight on the positive class and has a baseline equal to the positive rate (0.349) rather than 0.5:

$$\text{AUC} = \frac{1}{n_+ n_-} \sum_{i:\, y_i = 1} \sum_{j:\, y_j = 0} \mathbf{1}[\hat{p}_i > \hat{p}_j], \qquad \text{AP} = \sum_n (R_n - R_{n-1})\, P_n \tag{3.21}$$

The operating point is looked at with Youden's index [52], which is largest at the threshold that best balances sensitivity and specificity:

$$J(\tau) = \text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1 \tag{3.22}$$

The 50 repeated-CV scores share training rows, so a naive paired t-test underestimates the variance. The corrected resampled t-test of Nadeau and Bengio [53] inflates the variance of the per-split differences d_j by the test-to-training ratio:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{test}}{n_{train}}\right) s_d^2}}, \qquad J = 50, \quad \frac{n_{test}}{n_{train}} \approx \frac{154}{614} \tag{3.23}$$

Hold-out metrics are given with 95% percentile bootstrap intervals from 2,000 resamples of the test set [54]. Feature importance is measured by permutation [50]: the drop in hold-out ROC-AUC when one feature column is randomly shuffled, averaged over K = 30 repeats:

$$I_j = s(f,S) - \frac{1}{K}\sum_{k=1}^{K} s(f,S_{k,j}^{\pi}) \tag{3.24}$$

3.7 Algorithms

Algorithm 1 gives the leakage-safe training and evaluation procedure, Algorithm 2 the training loop of the ANN, and Algorithm 3 the inference step for a new record.

Algorithm 1: Leakage-safe stratified cross-validation of a pipeline

Input: data $D = \{(x_i, y_i)\}$, $i = 1..N$; classifier g; folds $k = 5$; seed $s = 42$
$Z \leftarrow$ {Glucose, BloodPressure, SkinThickness, Insulin, BMI}
Split indices into k stratified folds $S_1, \ldots, S_k$ using seed s
for $f = 1$ **to** k **do**
    $T \leftarrow \{1..N\} \setminus S_f$ // training indices; S_f is never used for fitting
    Recode zeros in columns Z of rows T and S_f as missing (fixed rule, Eq. 3.3)
    $m \leftarrow$ column medians of rows T; fill missing values of T and S_f with m (Eq. 3.4)
    if selection enabled **then** keep the k columns with the largest F-scores on T (Eq. 3.5)
    $\mu, \sigma \leftarrow$ mean and std of rows T; standardise T and S_f with μ, σ (Eq. 3.6)
    $T' \leftarrow$ random oversampling of the minority class in T (Eq. 3.7)
    Train g on T' (Algorithm 2 for the ANN)
    $\hat{p}_i \leftarrow g(z_i)$ and $\hat{y}_i \leftarrow 1[\hat{p}_i \geq 0.5]$ for every row i of S_f
    Store fold metrics (Eqs. 3.18–3.21) and out-of-fold $\hat{p}_i$
end for
return mean ± std of fold metrics, out-of-fold confusion matrix, ROC and PR curves

Algorithm 2: Training the ANN with Adam and early stopping

Input: standardised, resampled training set T'; $\alpha = 10^{-3}$; $\eta = 10^{-3}$; batch 32; patience 20
Split T' into training part T_a (85%) and validation part V (15%), stratified
Initialise $W^{(l)}$ by Glorot uniform (Eq. 3.10), $b^{(l)} \leftarrow 0$; best $\leftarrow -\infty$; wait $\leftarrow 0$
for epoch = 1 **to** 500 **do**
    Shuffle T_a and partition it into mini-batches B of 32 rows
    for each mini-batch B **do**
        Forward pass: $h^{(l)}$, $a^{(l)}$, $\hat{p}$ (Eqs. 3.8–3.9); loss $L(\theta)$ (Eq. 3.11)
        Backward pass: $\delta^{(3)}$, $\delta^{(2)}$, $\delta^{(1)}$ and gradients (Eqs. 3.12–3.13)
        Adam update of every $W^{(l)}$, $b^{(l)}$ (Eqs. 3.14–3.15)
    end for
    acc $\leftarrow$ accuracy of the network on V
    if acc > best + 10^{-4} **then** best $\leftarrow$ acc; $\theta_{best} \leftarrow \theta$; wait $\leftarrow 0$ **else** wait $\leftarrow$ wait + 1
    if wait = 20 **then break**
end for
return θ_{best} (weights of the best validation epoch)

Algorithm 3: Inference for a new record

Input: raw record x of 8 values; fitted objects (Z, m, μ, σ, θ_{best})
for each column j in Z **do if** $x_j = 0$ **then** $x_j \leftarrow m_j$ // impossible zero treated as missing
for $j = 1$ **to** 8 **do if** x_j is missing **then** $x_j \leftarrow m_j$
$z \leftarrow (x - \mu) / \sigma$
$\hat{p} \leftarrow \sigma(W^{(3)}\,\mathrm{ReLU}(W^{(2)}\,\mathrm{ReLU}(W^{(1)} z + b^{(1)}) + b^{(2)}) + b^{(3)})$
return $\hat{p}$ and $\hat{y} = 1[\hat{p} \geq 0.5]$ // educational risk estimate, not a diagnosis

3.8 Implementation Details

We wrote it in Python 3.9. scikit-learn 1.5.2 [55], imbalanced-learn [43], NumPy 2.0 [56], pandas 2.3 [57], Matplotlib 3.9 [58]. Small package: load, profile, preprocess, train, score, plot. Twenty unit tests. The ones we actually cared about: Pregnancies = 0 is never recoded, and imputation statistics come from the training fold only. Seed 42 everywhere. Table 3.1 lists the hyperparameters. None of them was tuned on a test fold.

Table 3.1: Hyperparameters and settings of the proposed system and baselines

Component	Setting
Zero recoding	Glucose, BloodPressure, SkinThickness, Insulin, BMI: 0 → missing; Pregnancies kept
Imputation / scaling	Median imputation; z-score standardisation (training-fold statistics)
Oversampling	RandomOverSampler to a 1:1 class ratio, training fold only
ANN architecture	8 → 16 (ReLU) → 8 (ReLU) → 1 (sigmoid); 289 parameters
ANN training	Adam, $\eta = 0.001$, batch 32, $\alpha = 0.001$, ≤ 500 epochs, early stopping (15%, patience 20)
Logistic regression	L2, C = 1, liblinear, balanced class weights
Random forest	100 trees, max depth 6, min. 5 rows per leaf, balanced class weights
Feature selection	SelectKBest, ANOVA F, k = 5 (optional; ablation k = 3–8)
Validation	Stratified 5-fold (seed 42); 10 × 5 repeated CV; 80/20 hold-out

4. EXPERIMENTAL RESULTS AND DISCUSSIONS

4.1 Experimental Setup

Everything was run on an Apple M5 laptop CPU, with no GPU. The full suite — 13 seeded cross-validation configurations, 250 repeated-CV fits, 35 ablation settings with 15 fits each, plus the leakage, nested-CV, seed-stability, and hold-out runs — took about 3 minutes. Four evaluation protocols are used, each for a different job:

- **P1 — seeded 5-fold CV (seed 42):** the comparison of 13 configurations, out-of-fold ROC/PR curves, and confusion matrices.
- **P2 — 10 × 5 repeated stratified CV (50 splits):** headline estimates and corrected t-tests for the five main models.
- **P3 — 3 × 5 repeated stratified CV (15 splits):** the ablation and leakage experiments, with the same splits for every configuration.
- **P4 — stratified 80/20 hold-out (614/154 rows):** a single final test after all choices were fixed.

4.2 Dataset Details

Table 4.1 summarises the dataset and Table 4.2 describes each feature from the local file. There are 768 rows and 9 columns, with no duplicate rows and no empty cells. The only missingness is in the form of zero codes.

Table 4.1: Summary of the Pima Indians Diabetes dataset

Property	Value
Name	Pima Indians Diabetes Database (UCI extract, 1990)
Original owner	National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK)
Population	Women aged ≥ 21 years of Pima heritage, near Phoenix, Arizona, USA
Instances / predictors	768 / 8 numeric predictors + binary Outcome
Class distribution	Outcome 0: 500 (65.1%); Outcome 1: 268 (34.9%)
Outcome definition	2-hour post-load plasma glucose ≥ 200 mg/dL (WHO criterion)
Duplicates / empty cells	0 / 0 (missing values are coded as 0)
Majority-class accuracy	0.651 (floor any useful model must beat)

Table 4.2: Feature dictionary and statistics (medians and correlations after recoding zeros)

Feature	Description (unit)	Range (raw)	Zeros	Missing after recode	Median y = 0 / y = 1	r with y
Pregnancies	Number of pregnancies	0–17	111	0 (kept)	2 / 4	0.222
Glucose	2-h OGTT plasma glucose (mg/dL)	0–199	5	5 (0.7%)	107 / 140	0.495
BloodPressure	Diastolic blood pressure (mm Hg)	0–122	35	35 (4.6%)	70 / 74	0.171
SkinThickness	Triceps skinfold (mm)	0–99	227	227 (29.6%)	27 / 32	0.259
Insulin	2-h serum insulin (µU/mL)	0–846	374	374 (48.7%)	102 / 170	0.303
BMI	Body-mass index (kg/m²)	0.0–67.1	11	11 (1.4%)	30.1 / 34.3	0.314
Pedigree (DPF)	Family-history score	0.078–2.420	0	0 (kept)	0.340 / 0.450	0.174
Age	Age (years)	21–81	0	0 (kept)	27 / 36	0.238

Figure 4.1 shows the class balance. Because 65.1% of rows are negative, accuracy on its own cannot tell a useful model from one that simply ignores the minority class. Figure 4.2 shows the zeros. Insulin has 374 of them (48.7% of rows) and SkinThickness 227 (29.6%). BloodPressure (35), BMI (11), and Glucose (5) have fewer. Those zeros are impossible in a living adult, so they are treated as missing. The 111 zeros in Pregnancies are real (never pregnant) and are kept. Recoding them would throw away information; Chapter 5 checks that this choice is right.

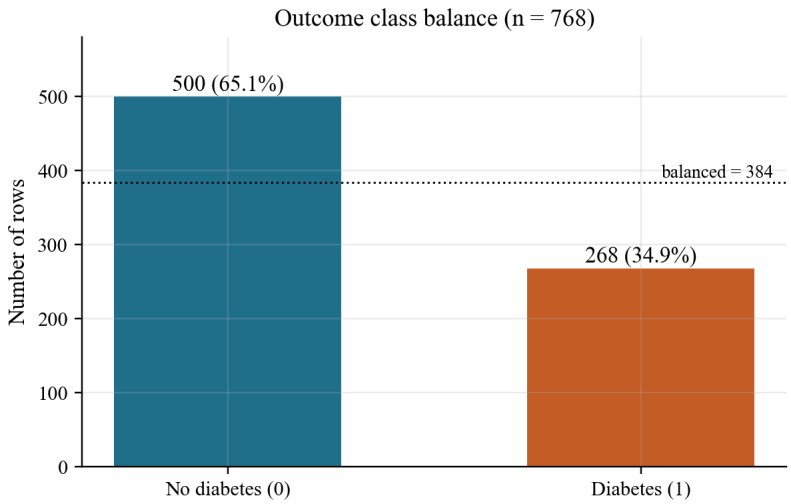


Figure 4.1: Outcome class balance of the 768 rows.

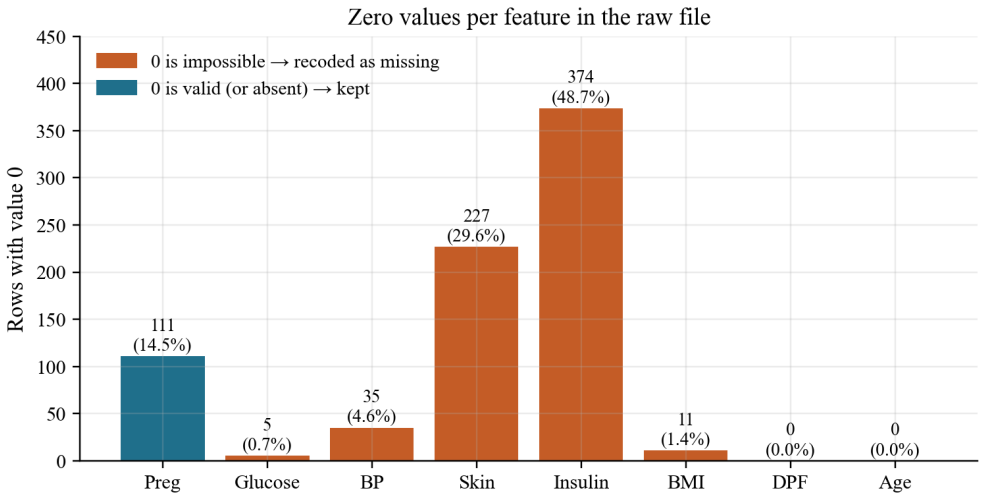


Figure 4.2: Zero values per feature in the raw file; orange bars are recoded as missing, blue bars are kept.

4.3 Exploratory Data Analysis

Figure 4.3 compares each feature in the two classes after recoding. Glucose is the clearest split: the median is 107 mg/dL for negative rows and 140 mg/dL for positive ones. Positive rows also have higher BMI (median 30.1 vs 34.3), are older (median 27 vs 36 years), and have more pregnancies. Insulin and skinfold separate the classes once zeros are removed (mean insulin 130 vs 207 µU/mL); in the raw file the zeros hide that. Blood pressure and the pedigree function overlap a lot between the two classes.

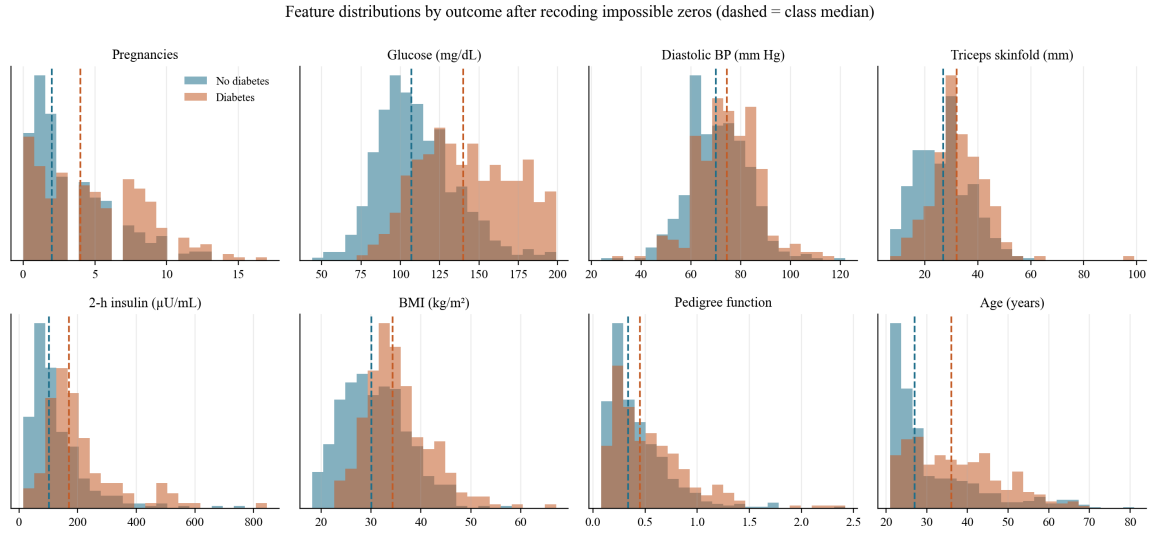


Figure 4.3: Distribution of each feature by outcome after recoding impossible zeros; dashed lines mark class medians.

The box plots in Figure 4.4 show long right tails for insulin, the pedigree function, and age, plus a few extreme BMI and skinfold values. The raw 1.5 × IQR counts are modest — 34 for insulin and 29 for the pedigree function, for example. Those points were looked at but not deleted. Dropping them on the full table would use test-fold information and

shrink an already small dataset, and the median imputer and the small regularised network are not very sensitive to a few extremes.

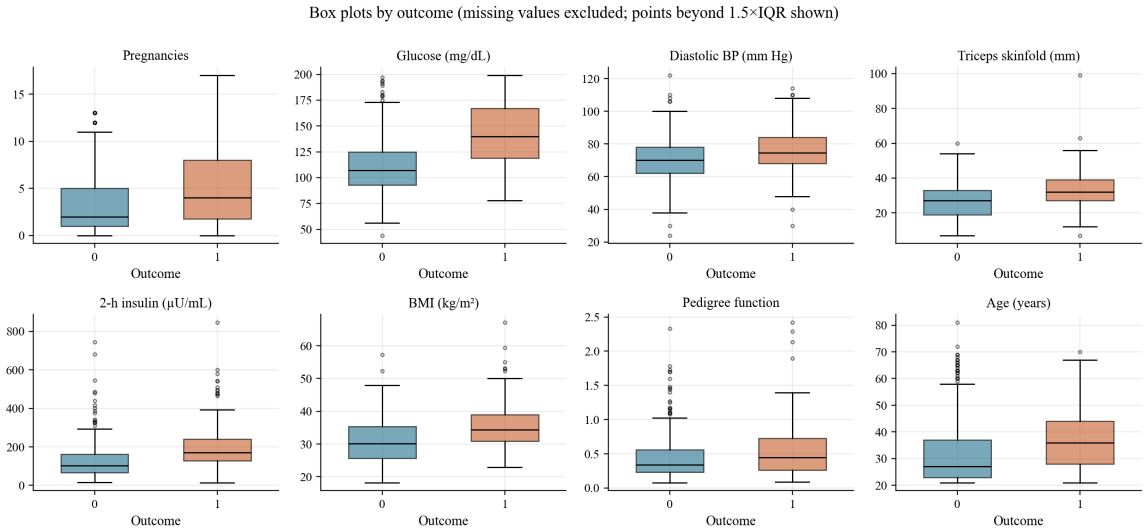


Figure 4.4: Box plots of each feature by outcome after recoding (points beyond 1.5 × IQR shown).

Figure 4.5 shows Pearson correlations after recoding. Glucose is most associated with the outcome (r = 0.495), then BMI (0.314), insulin (0.303), skinfold (0.259), and age (0.238). Among the predictors, skinfold and BMI (r ≈ 0.65), glucose and insulin (r ≈ 0.58), and age and pregnancies (r ≈ 0.54) are the most redundant pairs. None of those pairs is so strong that a feature has to be dropped, and no single feature explains the outcome on its own, so a multivariate model is still needed.

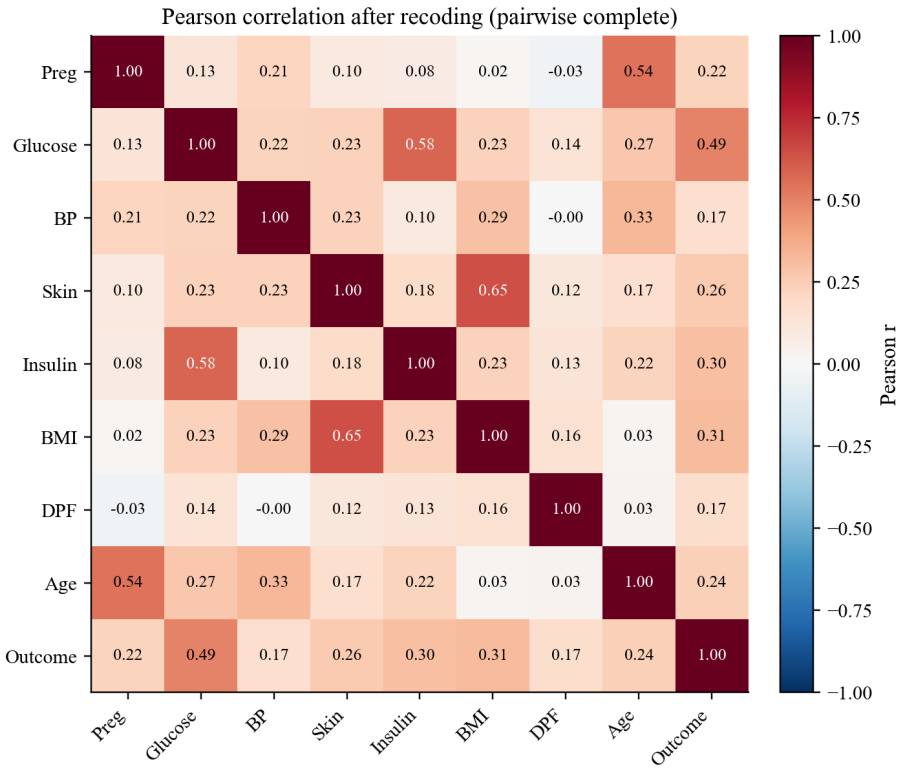


Figure 4.5: Pearson correlation matrix after recoding zeros (pairwise complete observations).

4.4 Feature Relevance and Selection

Figure 4.6 (left) shows ANOVA F-scores computed separately on each training fold. Glucose dominates (F ≈ 197), then BMI (≈ 66), age (≈ 37), and pregnancies (≈ 32). The right panel shows how often SelectKBest with k = 5 kept each feature: glucose, BMI, and age in all five folds, skinfold in four, pregnancies and insulin in three, and blood pressure and the pedigree function never. The selected set changes from fold to fold, which is what should happen when selection is refitted on each training fold, and it is a check that the set was not fixed in advance on all rows.

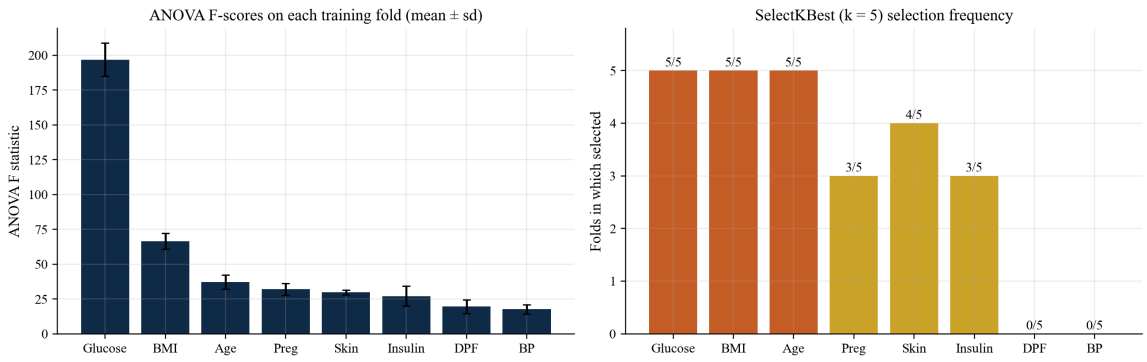


Figure 4.6: ANOVA F-scores on each training fold (left) and SelectKBest (k = 5) selection frequency across the five folds (right).

4.5 Training Behaviour of the ANN

Figure 4.7 plots training loss and early-stopping validation accuracy on the first fold. Loss falls smoothly from 0.79 to 0.43. Validation accuracy rises quickly, peaks at 0.758 in epoch 15, then levels off. Training stopped at epoch 36 after 20 epochs with no improvement, and the best weights were restored. The smooth curve suggests the learning rate and batch size are reasonable; the plateau says extra epochs would mostly fit noise.

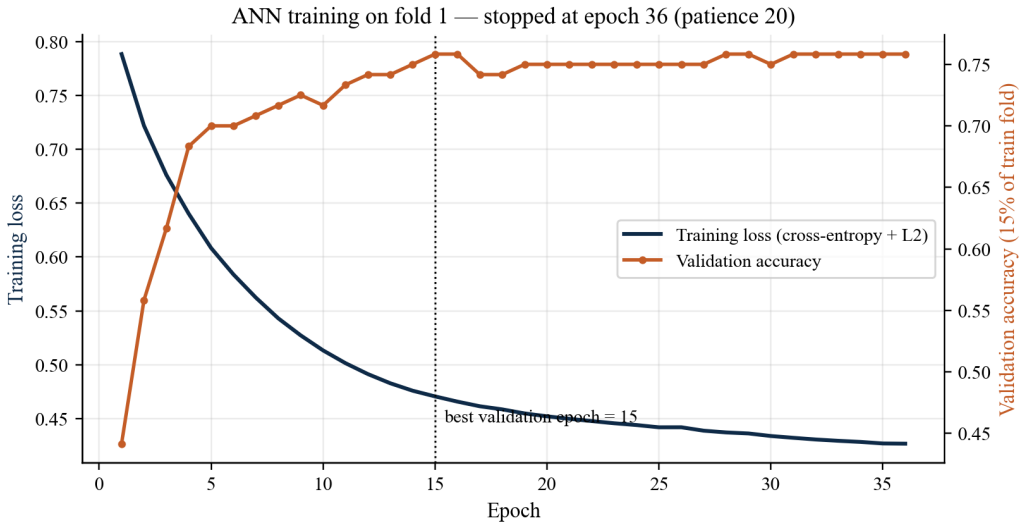


Figure 4.7: Training loss and early-stopping validation accuracy of the ANN on fold 1.

The learning curve in Figure 4.8 shows validation ROC-AUC rising from 0.799 with 122 training rows to 0.832 with 614 rows, while training ROC-AUC stays near 0.85. The gap is small (about 0.02–0.05), so the regularised network is not over-fitting badly. The validation curve is still rising, which points to more data — not a bigger network — as the main way to do better.

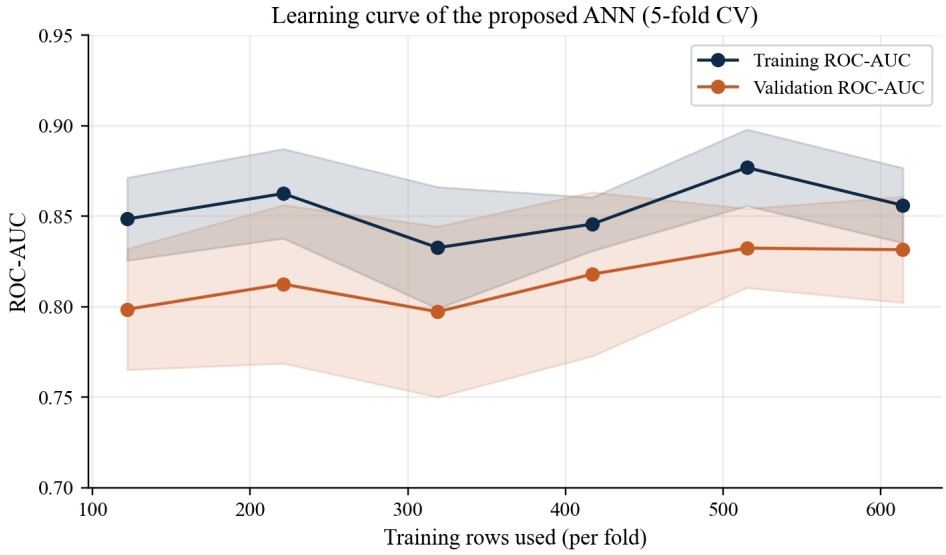


Figure 4.8: Learning curve of the proposed ANN (mean ± sd over five folds).

4.6 Cross-Validated Comparison of All Configurations

Table 4.3 has protocol P1 results for all 13 configurations: the dummy floor, plus logistic regression, random forest, and the ANN, each with all eight features or SelectKBest (k = 5), and each with and without training-fold oversampling. Figure 4.9 is the console output of this run, with the same numbers and their fold standard deviations.

Table 4.3: Protocol P1: seeded 5-fold cross-validation of all configurations (mean over folds; bold = proposed system)

Model	Inputs	ROS	Acc.	Prec.	Sens.	Spec.	F1	ROC-AUC	PR-AUC
Dummy (majority)	—	—	0.651	0.000	0.000	1.000	0.000	0.500	0.349
Logistic regression	All 8	No	0.747	0.623	0.727	0.758	0.669	0.836	0.725
Logistic regression	All 8	Yes	0.742	0.619	0.709	0.760	0.659	0.834	0.719
Logistic regression	k = 5	No	0.737	0.613	0.690	0.762	0.648	0.826	0.711
Logistic regression	k = 5	Yes	0.737	0.615	0.686	0.764	0.646	0.827	0.715
Random forest	All 8	No	0.766	0.636	0.780	0.758	0.700	0.837	0.712
Random forest	All 8	Yes	0.759	0.631	0.769	0.754	0.691	0.832	0.702
Random forest	k = 5	No	0.766	0.640	0.769	0.764	0.697	0.837	0.717
Random forest	k = 5	Yes	0.762	0.633	0.776	0.754	0.696	0.840	0.725
MLP ANN	All 8	No	0.762	0.698	0.571	0.864	0.625	0.832	0.699
MLP ANN	**All 8**	**Yes**	**0.746**	**0.613**	**0.769**	**0.734**	**0.680**	**0.834**	**0.696**
MLP ANN	k = 5	No	0.741	0.657	0.556	0.840	0.600	0.789	0.671
MLP ANN	k = 5	Yes	0.751	0.622	0.750	0.752	0.678	0.842	0.727

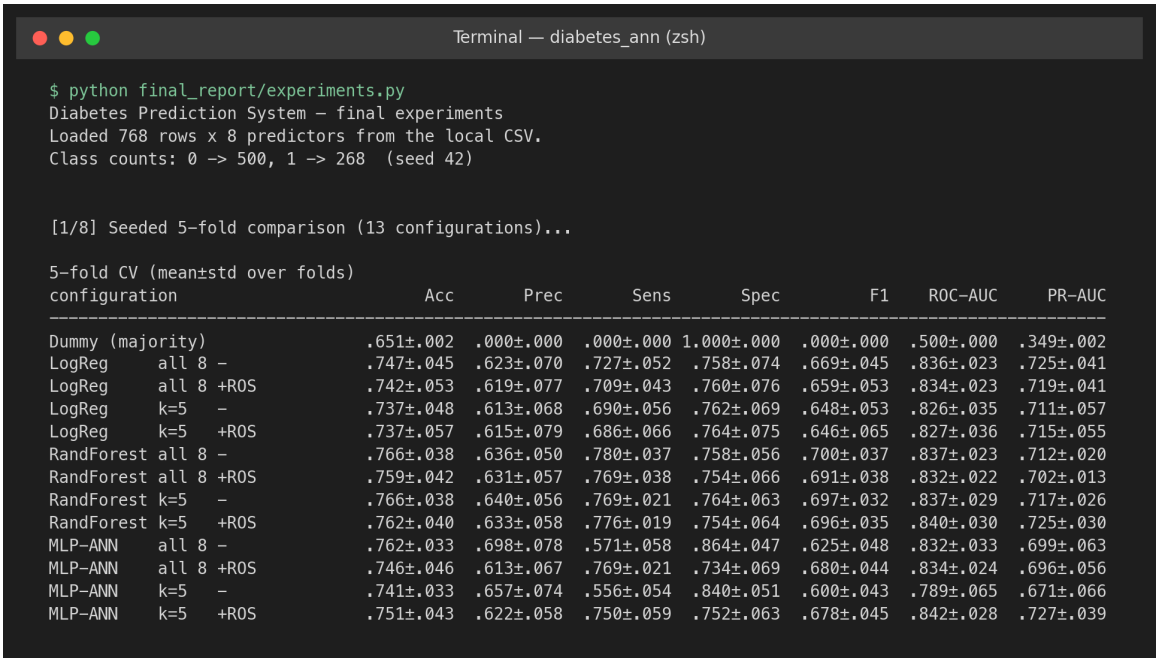


Figure 4.9: Console output of the experiment script: data loading and the seeded 5-fold comparison of 13 configurations (mean ± sd over folds).

A few things jumped out. Every trained model beats the dummy on ROC-AUC (0.79–0.84 versus 0.50). The three families then sit in a tight band (0.826–0.842). Ranking, on this table, is mostly a property of the data. The ANN trained on the raw imbalance is shy: highest specificity (0.864) but it only finds 57.1% of the diabetic rows. Oversample the training fold and sensitivity goes to 0.769. F1 goes from 0.625 to 0.680. That is why we kept oversampling. Logistic regression and the forest already use class weights, so oversampling barely touches them.

4.7 Repeated Cross-Validation and Statistical Significance

A single 5-fold run moves around with the split, so the five main models were re-run under protocol P2. Table 4.4 gives mean ± standard deviation over 50 splits, and Table 4.5 the corrected resampled t-tests.

Table 4.4: Protocol P2: 10 × 5 repeated stratified cross-validation (mean ± sd over 50 splits)

Metric	Dummy	Logistic reg.	Random forest	ANN (no ROS)	Proposed ANN
Accuracy	0.651 ± 0.002	0.754 ± 0.034	0.761 ± 0.030	0.760 ± 0.033	0.746 ± 0.035
Precision	0.000 ± 0.000	0.632 ± 0.047	0.632 ± 0.041	0.695 ± 0.063	0.610 ± 0.047
Sensitivity	0.000 ± 0.000	0.718 ± 0.064	0.761 ± 0.048	0.560 ± 0.080	0.768 ± 0.050
Specificity	1.000 ± 0.000	0.774 ± 0.042	0.761 ± 0.039	0.867 ± 0.035	0.734 ± 0.052
F1-score	0.000 ± 0.000	0.671 ± 0.046	0.690 ± 0.037	0.617 ± 0.063	0.679 ± 0.038
ROC-AUC	0.500 ± 0.000	0.836 ± 0.027	0.838 ± 0.028	0.830 ± 0.030	0.834 ± 0.028
PR-AUC	0.349 ± 0.002	0.722 ± 0.048	0.717 ± 0.051	0.704 ± 0.054	0.711 ± 0.056

Table 4.5: Corrected resampled t-tests (Nadeau–Bengio) of the proposed ANN against each comparator

Comparison (proposed − other)	Δ ROC-AUC (p)	Δ PR-AUC (p)	Δ F1 (p)	Δ Sensitivity (p)
vs Logistic regression	-0.002 (0.898)	-0.011 (0.614)	+0.008 (0.709)	+0.050 (0.069)
vs Random forest	-0.004 (0.739)	-0.006 (0.798)	-0.011 (0.592)	+0.007 (0.787)
vs MLP ANN (no rebalancing)	+0.004 (0.634)	+0.007 (0.688)	+0.062 (0.041)	+0.208 (<0.001)

The proposed ANN reaches ROC-AUC 0.834 ± 0.028, basically the same as logistic regression (0.836 ± 0.027; Δ = -0.002, p = 0.898) and the random forest (0.838 ± 0.028; p = 0.739). None of those ranking gaps is statistically significant. At the default threshold the ANN has the highest sensitivity of the trained models (0.768), 0.050 above logistic regression (p = 0.069), with a matching drop in specificity. The one clear effect is oversampling: against the same network on imbalanced folds, sensitivity rises by 0.208 (p < 0.001) and F1 by 0.062 (p = 0.041), while ROC-AUC stays put. Oversampling therefore moves the decision boundary; it does not make the ranking better. Figure 4.10 plots these scores, and Figure 4.11 is the console output the tables were taken from.

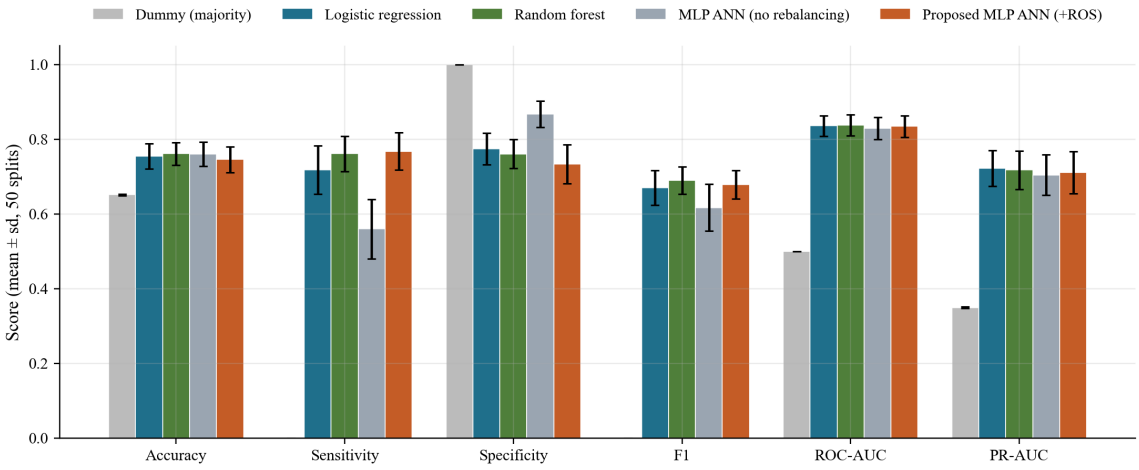


Figure 4.10: Repeated cross-validation scores of the five main models (mean ± sd over 50 splits).

```
Terminal — diabetes_ann (zsh)

$ python final_report/experiments.py   # (continued)
[2/8] 10 x 5 repeated stratified CV for the main models...

Repeated CV, 50 train/test splits (mean±std)
configuration                       Acc       Prec       Sens       Spec         F1    ROC-AUC     PR-AUC
-------------------------------------------------------------------------------------------------------------
Dummy (majority)                .651±.002  .000±.000  .000±.000 1.000±.000  .000±.000  .500±.000  .349±.002
Logistic regression             .754±.034  .632±.047  .718±.064  .774±.042  .671±.046  .836±.027  .722±.048
Random forest                   .761±.030  .632±.041  .761±.048  .761±.039  .690±.037  .838±.028  .717±.051
MLP ANN (no rebalancing)        .760±.033  .695±.063  .560±.080  .867±.035  .617±.063  .830±.030  .704±.054
Proposed MLP ANN (+ROS)         .746±.035  .610±.047  .768±.050  .734±.052  .679±.038  .834±.028  .711±.056

Corrected resampled t-tests, proposed minus comparator (difference, p-value):
  vs Logistic regression       ROC-AUC -0.002 (p=0.898)  F1 +0.008 (p=0.709)  Sens +0.050 (p=0.069)
  vs Random forest             ROC-AUC -0.004 (p=0.739)  F1 -0.011 (p=0.592)  Sens +0.007 (p=0.787)
  vs MLP ANN (no rebalancing)  ROC-AUC +0.004 (p=0.634)  F1 +0.062 (p=0.041)  Sens +0.208 (p=0.000)
```

Figure 4.11: Console output of the repeated cross-validation and the corrected t-tests.

4.8 ROC, Precision–Recall, and Confusion Matrices

Figure 4.12 shows out-of-fold ROC and precision–recall curves under protocol P1. The four trained models' curves sit on top of each other for most of the range, so their ranking ability is similar. The precision–recall panel is harsher: precision drops below 0.6 once recall goes past about 0.8 for every model. Finding most of the diabetic cases without a pile of false alarms is hard on this table. The dotted line is the PR baseline of 0.349, the positive rate.

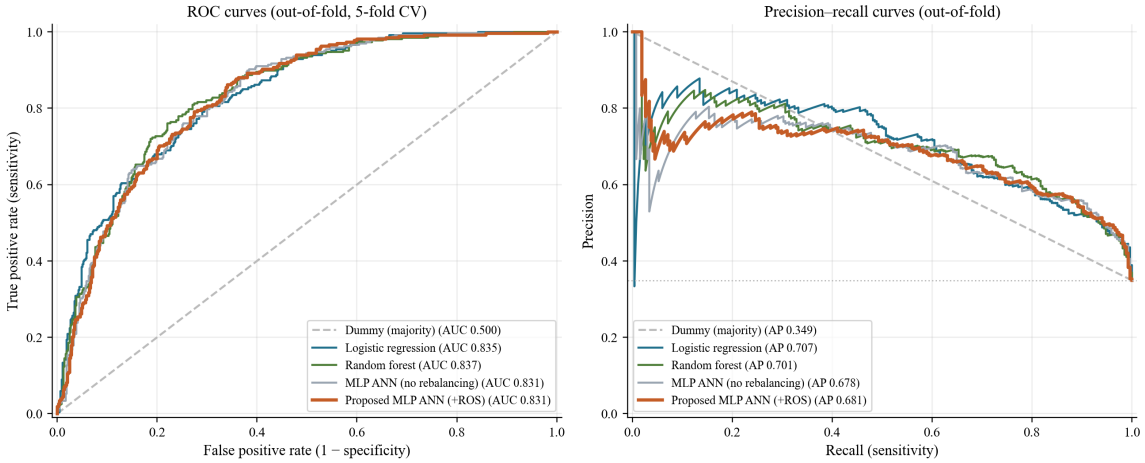


Figure 4.12: Out-of-fold ROC curves (left) and precision–recall curves (right) under seeded 5-fold cross-validation.

The out-of-fold confusion matrices in Figure 4.13 make those operating points concrete. The ANN trained without rebalancing misses 115 of 268 diabetic cases. The proposed ANN with training-fold oversampling misses 62 and finds 206, at the cost of 133 false alarms among 500 negatives instead of 68. If a missed case is more costly than an extra confirmatory test — which is the usual screening trade-off — the proposed operating point is the more useful one.

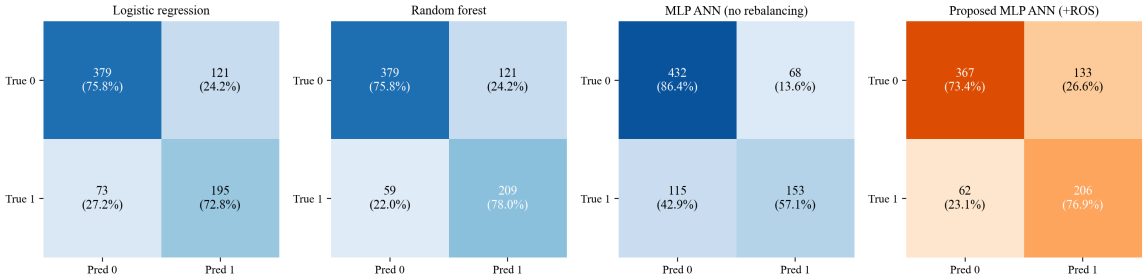


Figure 4.13: Out-of-fold confusion matrices (counts and row percentages) of the four trained models.

4.9 Decision-Threshold Analysis

Figure 4.14 sweeps the decision threshold τ over the proposed ANN's out-of-fold probabilities. Sensitivity falls and specificity rises as τ goes up, while F1 stays between 0.66 and 0.69 for τ from 0.3 to 0.6. Youden's index peaks at $\tau = 0.40$ (sensitivity 0.862, specificity 0.652). That threshold was read off the evaluation predictions, so it is a bit optimistic. It is shown only to make the trade-off visible; the headline results keep the pre-declared $\tau = 0.5$. In practice the threshold should be set from the relative costs of missed cases and false alarms, on data that were not used for evaluation.

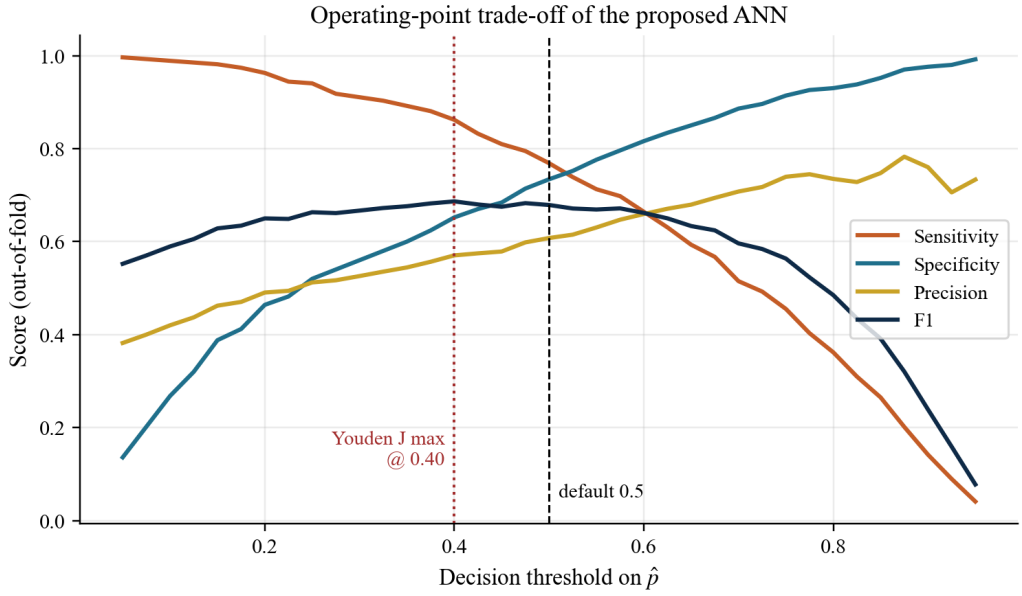


Figure 4.14: Sensitivity, specificity, precision, and F1 of the proposed ANN as a function of the decision threshold (out-of-fold).

4.10 Hold-Out Test Evaluation

Protocol P4 trains each model once on 614 rows and scores it once on 154 untouched rows (54 of them positive). Table 4.6 has the numbers, and Figure 4.15 has the ROC curves plus the proposed ANN's confusion matrix.

Table 4.6: Protocol P4: stratified 80/20 hold-out test (154 rows) with bootstrap 95% intervals for the proposed ANN

Model	Acc.	Prec.	Sens.	Spec.	F1	ROC-AUC	PR-AUC
Logistic regression	0.734	0.603	0.704	0.750	0.650	0.813	0.673
Random forest	0.734	0.600	0.722	0.740	0.655	0.821	0.698
MLP ANN (no rebalancing)	0.727	0.615	0.593	0.800	0.604	0.807	0.645
Proposed MLP ANN (+ROS)	**0.747**	**0.606**	**0.796**	**0.720**	**0.688**	**0.821**	**0.679**
Proposed ANN, 95% CI	0.68–0.81	—	0.68–0.90	0.63–0.80	0.59–0.78	0.75–0.88	—

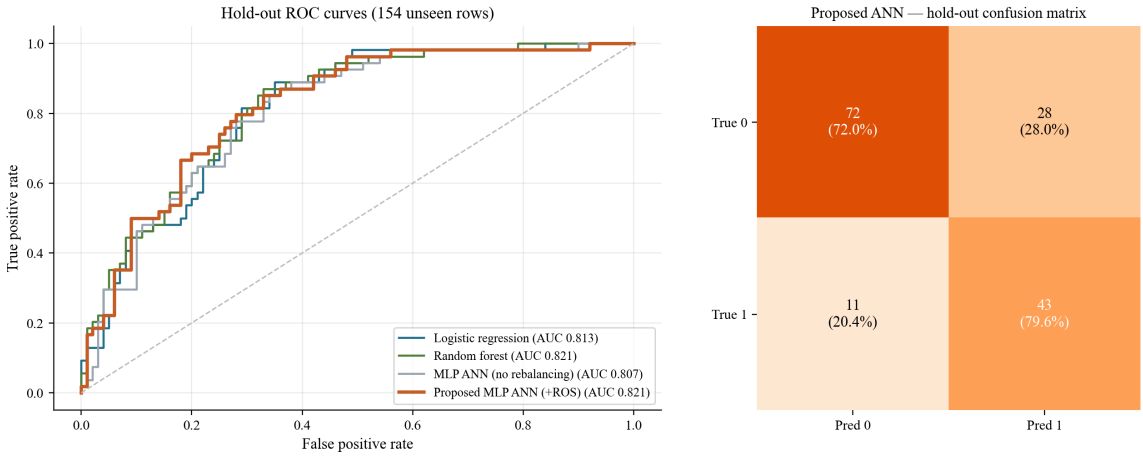


Figure 4.15: Hold-out ROC curves of the four trained models (left) and the confusion matrix of the proposed ANN (right).

On the hold-out set the proposed ANN has the highest accuracy (0.747), sensitivity (0.796), and F1 (0.688) of the four models, and it ties the random forest on ROC-AUC (0.821). It correctly flags 43 of 54 diabetic patients and 72 of 100 non-diabetic patients. The bootstrap intervals are wide (sensitivity 0.68–0.90, for example) because there are only 54 positives, so the small gaps between models on this one split are not conclusive. The repeated-CV results in Section 4.7 are the safer comparison. The hold-out numbers still sit close to the cross-validated estimates, which is a check that the CV protocol is not painting a rosy picture.

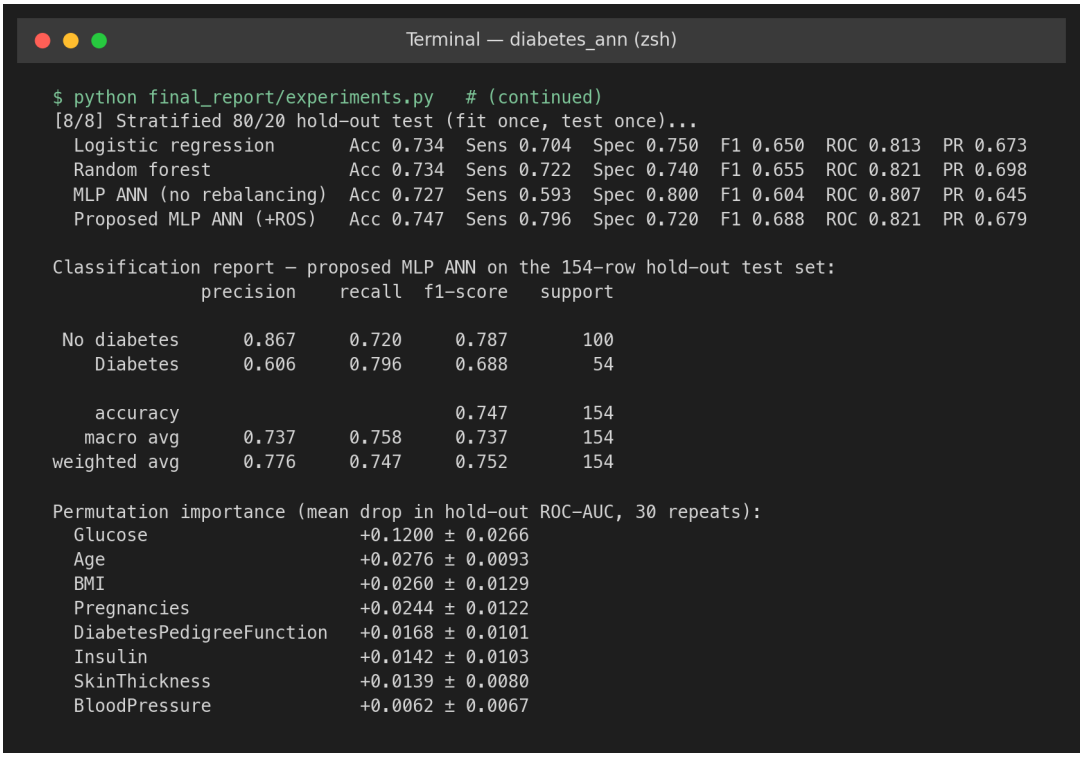


Figure 4.16: Console output of the hold-out evaluation: per-model metrics, the classification report of the proposed ANN, and permutation importances.

The classification report in Figure 4.16 breaks the same result down by class. For the diabetic class, precision is 0.606 and recall 0.796; for the non-diabetic class, precision is 0.867 and recall 0.720. The model is better at ruling diabetes out (a negative call is right about 87% of the time) than at confirming it. That is what a screening-style operating point is supposed to do.

4.11 Interpretability: Permutation Importance

Figure 4.17 shows how much the hold-out ROC-AUC drops when each feature is shuffled. Glucose is clearly the most important input (drop 0.120), then age (0.028), BMI (0.026), and pregnancies (0.024). Blood pressure contributes least (0.006). That order matches the ANOVA scores, the selection frequencies, the correlations, and the predictors picked by Joshi and Dhakal [28] and Howlader et al. [41]. It is a useful check that the network is using signals a clinician would recognise, not artefacts of the table.

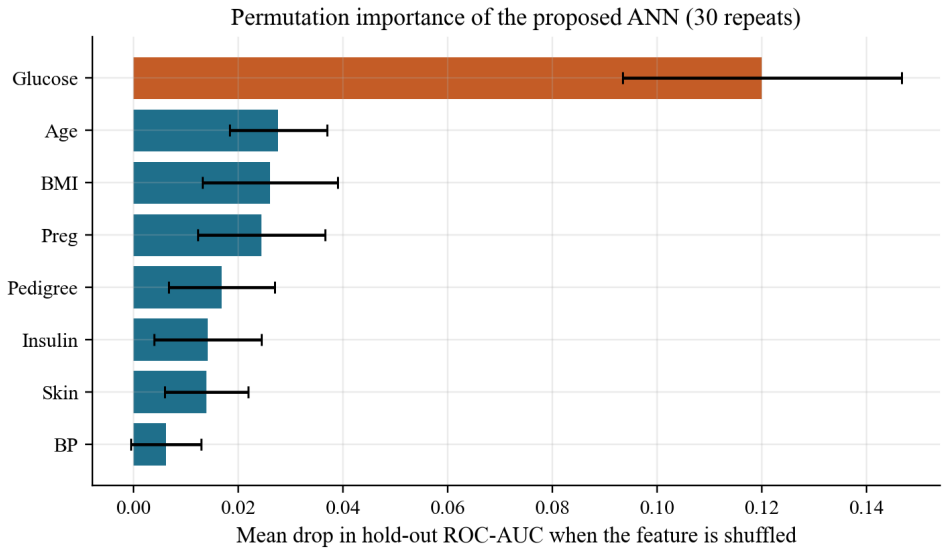


Figure 4.17: Permutation importance of the proposed ANN on the hold-out set (mean ± sd over 30 shuffles).

4.12 System Output Demonstration

Figure 4.18 shows the prediction module in use. The saved pipeline (zero recoding → median imputation → scaling → oversampling → ANN; oversampling is idle at prediction time) is loaded and applied to the first ten hold-out patients. It prints the inputs, the predicted probability P(diabetes), the predicted class, the true class, and whether the call was right. Of those ten rows, six are correct, three are false alarms, and one diabetic case is missed. Across all 154 hold-out rows the module is right for 115 (74.7%). Two made-up records follow: a 47-year-old with glucose 168 mg/dL and BMI 36.4 gets P = 0.914; a 26-year-old with glucose 94 mg/dL and BMI 24.1 gets P = 0.018. Zeros entered for skinfold and insulin in the first record are treated as missing and filled in, the same way as in training.

```
Terminal — diabetes_ann (zsh)

$ python final_report/predict_demo.py
Proposed ANN: 8 -> 16 (ReLU) -> 8 (ReLU) -> 1 (sigmoid), trained on 614 rows
Loaded pipeline steps: zero_as_missing -> impute -> scale -> resample -> clf

 Row   Preg   Gluc     BP   Skin   Insu    BMI    DPF    Age  P(diab)  Pred  True  Result
-------------------------------------------------------------------------------------------
  44      7    159     64      0      0   27.4  0.294     40    0.882     1     0  false alarm
 672     10     68    106     23     49   35.5  0.285     47    0.058     0     0  correct
 700      2    122     76     27    200   35.9  0.483     26    0.675     1     0  false alarm
 630      7    114     64      0      0   27.4  0.732     34    0.645     1     1  correct
  81      2     74      0      0      0    0.0  0.102     22    0.020     0     0  correct
 389      3    100     68     23     81   31.6  0.949     28    0.421     0     0  correct
 387      8    105    100     36      0   43.3  0.239     45    0.433     0     1  missed case
 408      8    197     74      0      0   25.9  1.191     39    0.871     1     1  correct
 163      2    100     64     23      0   29.7  0.368     21    0.062     0     0  correct
 335      0    165     76     43    255   47.9  0.259     26    0.927     1     0  false alarm

Hold-out rows scored: 154; correct at threshold 0.5: 115 (74.7%)

Illustrative new inputs (zeros in Skin/Insulin are treated as missing):
  Record 1: P(diabetes) = 0.914 -> Outcome = 1 (higher risk)
  Record 2: P(diabetes) = 0.018 -> Outcome = 0 (lower risk)

Educational model only — not a medical diagnostic tool. The Pima Indians Diabetes dataset is small (768 rows) and limited ...
```

Figure 4.18: Output of the prediction module on hold-out patients and two new records.

Figure 4.19 is the automated test suite. The 20 tests cover loading (ordinary, header-less, and alias-named CSV files, plus the ARFF converter), profiling of zeros and duplicates, the zero policy (Pregnancies never recoded), leakage safety (imputation statistics come from the training fold only, feature selection sits inside the pipeline, and resampling uses an imbalanced-learn pipeline so it runs only during fitting), metrics on edge cases, and an end-to-end run on a small table. All 20 pass.

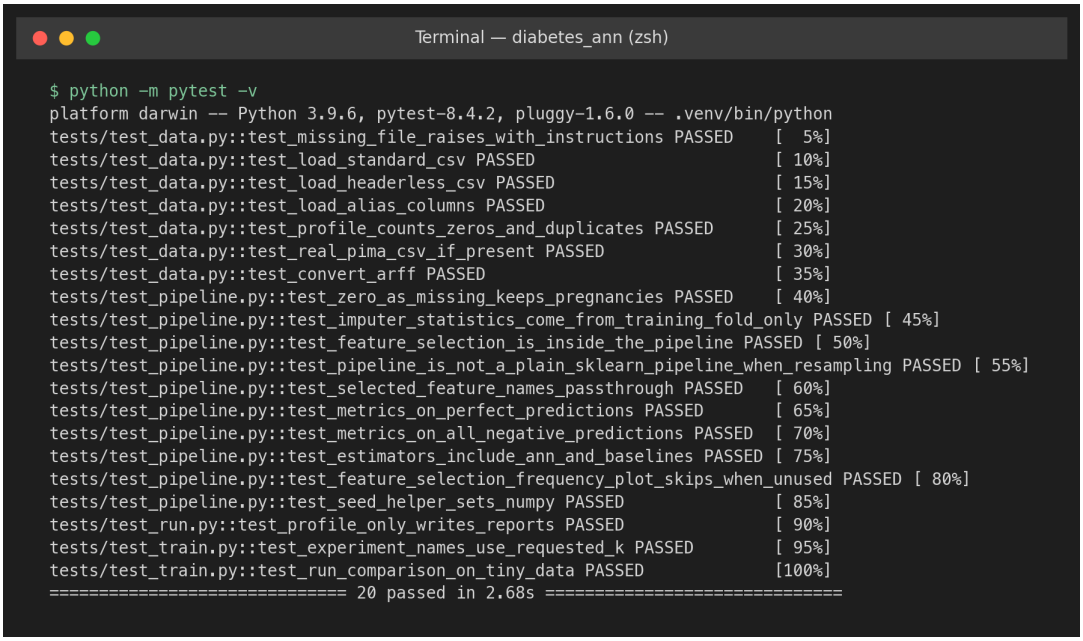


Figure 4.19: Unit-test output (20 tests passed).

4.13 Discussion

That answers the questions from Chapter 1, more or less. A small regularised ANN, trained without leaking, sits around ROC-AUC 0.83. With training-fold oversampling the operating point is about sensitivity 0.77 and specificity 0.73. Same neighbourhood as the old 76%/76% ADAP cut. Indistinguishable from logistic regression and the forest on ranking. Extra ANN flexibility did not buy a better rank here. Glucose, BMI and age already carry most of the signal, and they are mostly monotonic. A linear model sees them. The useful bit is the screening-style cut, and that came from oversampling, not from the extra layers. The learning curve says the same thing. More rows would help. A fatter network probably would not.

5. ABLATION STUDY AND STATE-OF-THE-ART ANALYSIS

5.1 Ablation Protocol

Each ablation takes one piece out of the proposed system, or swaps it, and leaves everything else alone. All of them are scored with protocol P3 (3 × 5 repeated stratified CV, the same splits for every configuration). Gaps smaller than about one standard deviation (≈ 0.03–0.04 in ROC-AUC and F1) should be read as no clear effect. The ablation is descriptive: the proposed setup was fixed before these runs and was not re-tuned afterwards.

5.2 Component Ablation

Table 5.1 removes or replaces one pipeline component at a time, Figure 5.1 plots the resulting F1 and ROC-AUC, and Figure 5.2 shows the corresponding console output with standard deviations.

Table 5.1: Component ablation of the proposed pipeline (protocol P3, mean over 15 splits)

Configuration	Sens.	Spec.	F1	ROC-AUC	PR-AUC	Δ F1	Δ ROC-AUC
Proposed (full pipeline)	**0.772**	**0.737**	**0.684**	**0.837**	**0.706**	—	—
− zero recoding (zeros kept)	0.740	0.729	0.659	0.820	0.692	-0.025	-0.017
Recode Pregnancies = 0 as well	0.772	0.734	0.682	0.832	0.707	-0.002	-0.005
Median → mean imputation	0.777	0.740	0.688	0.839	0.711	+0.004	+0.003
Median → KNN imputation	0.765	0.723	0.672	0.826	0.691	-0.013	-0.010
− standard scaling	0.675	0.639	0.571	0.721	0.600	-0.113	-0.116
− oversampling (imbalanced)	0.553	0.861	0.608	0.831	0.700	-0.076	-0.005
ROS → SMOTE	0.772	0.739	0.684	0.836	0.707	+0.000	-0.000
+ SelectKBest (k = 5)	0.767	0.723	0.673	0.828	0.702	-0.011	-0.009
− early stopping	0.657	0.745	0.616	0.784	0.644	-0.068	-0.053

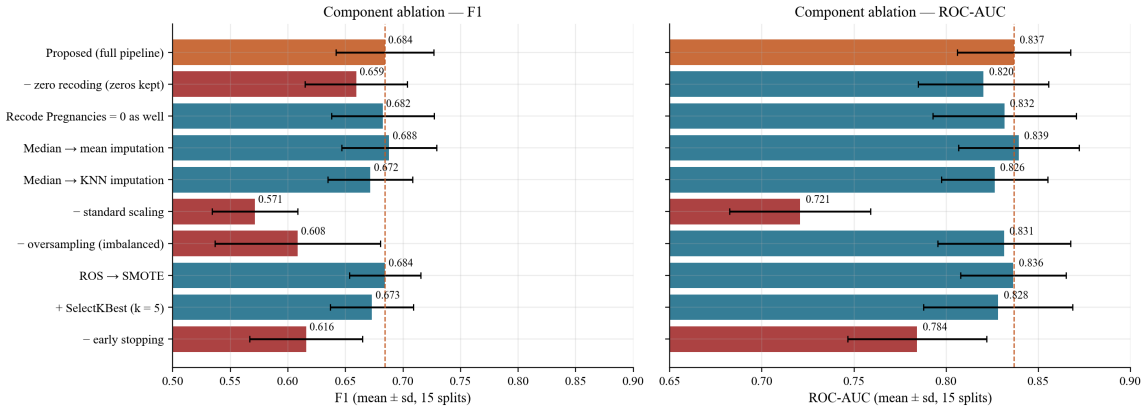


Figure 5.1: Component ablation: F1 (left) and ROC-AUC (right); red bars fall more than 0.02 below the proposed system.

Standardisation is the piece that matters most. Without it ROC-AUC falls from 0.837 to 0.721 and F1 from 0.684 to 0.571. Raw insulin values in the hundreds swamp the first-layer activations and the gradient steps, so the network barely sees small-scale inputs such as the pedigree function. **Early stopping** is next: training for the full 500 epochs lowers ROC-AUC to 0.784. That is over-fitting, as expected for a 289-parameter network on about 700 rows. **Zero recoding** also matters. Keeping the zeros as real values lowers ROC-AUC by 0.017 and F1 by 0.025, because a coded zero makes, for example, an insulin reading look like an extreme low.

Oversampling does not change ranking (ROC-AUC 0.831 without it) but it does set the operating point. Without it, sensitivity falls to 0.553 and F1 to 0.608. Swapping random oversampling for **SMOTE** gives almost the same numbers, so the simpler method is kept. **Mean imputation** does as well as the median (ROC-AUC 0.839). **KNN imputation** is a little worse (0.826), probably because its distances are computed on unscaled features. Recoding **Pregnancies = 0** as missing slightly lowers ROC-AUC (0.832), which is why valid zeros are kept. Adding **SelectKBest (k = 5)** drops ROC-AUC to 0.828. With only eight inputs, throwing three away loses some signal and gives the network nothing back.

```
Terminal — diabetes_ann (zsh)

$ python final_report/experiments.py   # (continued)
[3/8] Ablation study (15 splits per configuration)...

Ablation — component
configuration                          Acc        Prec       Sens       Spec       F1         ROC-AUC    PR-AUC
-------------------------------------------------------------------------------------------------------------------
Proposed (full pipeline)          .750±.043  .617±.061  .772±.039  .737±.063  .684±.042  .837±.031  .706±.063
− zero recoding (zeros kept)      .733±.041  .597±.054  .740±.054  .729±.059  .659±.044  .820±.035  .692±.058
Recode Pregnancies = 0 as well    .747±.046  .614±.063  .772±.045  .734±.069  .682±.045  .832±.039  .707±.072
Median → mean imputation          .753±.039  .619±.054  .777±.051  .740±.058  .688±.041  .839±.033  .711±.069
Median → KNN imputation           .738±.038  .601±.051  .765±.038  .723±.058  .672±.037  .826±.029  .691±.060
− standard scaling                .651±.043  .513±.063  .675±.127  .639±.122  .571±.037  .721±.038  .600±.055
− oversampling (imbalanced)       .754±.038  .684±.077  .553±.088  .861±.041  .608±.072  .831±.036  .700±.069
ROS → SMOTE                       .750±.035  .617±.051  .772±.040  .739±.060  .684±.031  .836±.029  .707±.063
+ SelectKBest (k = 5)             .739±.038  .602±.053  .767±.041  .723±.060  .673±.036  .828±.040  .702±.071
− early stopping                  .714±.040  .583±.058  .657±.061  .745±.055  .616±.049  .784±.038  .644±.059
```

Figure 5.2: Console output of the component ablation (mean ± sd over 15 splits).

5.3 Architecture and Hyperparameter Ablation

Table 5.2 and Figure 5.3 change one property of the network at a time (hidden layers, activation, optimiser, L2 penalty, and number of selected features) while the rest of the proposed pipeline stays put.

Table 5.2: Architecture and hyperparameter ablation (protocol P3; * = setting used by the proposed system; k = 8 equals no selection)

Group	Setting	Sens.	Spec.	F1	ROC-AUC	PR-AUC
Hidden layers	8	0.765	0.711	0.665	0.824	0.703
	16	0.753	0.759	0.684	0.829	0.708
	32	0.756	0.758	0.686	0.836	0.713
	16 → 8 *	**0.772**	**0.737**	**0.684**	**0.837**	**0.706**
	32 → 16	0.745	0.758	0.678	0.835	0.715
	64 → 32	0.753	0.755	0.681	0.831	0.692
	64 → 32 → 16	0.729	0.746	0.662	0.821	0.686
Activation	**relu ***	**0.772**	**0.737**	**0.684**	**0.837**	**0.706**
	tanh	0.774	0.723	0.676	0.835	0.707
	logistic	0.760	0.701	0.657	0.813	0.679
Optimiser	**Adam ***	**0.772**	**0.737**	**0.684**	**0.837**	**0.706**
	SGD + momentum	0.779	0.746	0.692	0.836	0.712
	L-BFGS	0.591	0.763	0.581	0.754	0.573
L2 penalty α	1e-05	0.768	0.736	0.680	0.837	0.708
	0.0001	0.775	0.735	0.684	0.837	0.707
	0.001 *	**0.772**	**0.737**	**0.684**	**0.837**	**0.706**
	0.01	0.767	0.739	0.682	0.836	0.706
	0.1	0.767	0.736	0.680	0.838	0.710
	1	0.786	0.704	0.674	0.836	0.706
Features kept k	3	0.772	0.717	0.672	0.828	0.704
	4	0.739	0.728	0.658	0.825	0.711
	5	0.767	0.723	0.673	0.828	0.702
	6	0.759	0.730	0.671	0.829	0.714
	7	0.782	0.685	0.660	0.817	0.688
	8 *	0.772	0.737	0.684	0.837	0.706

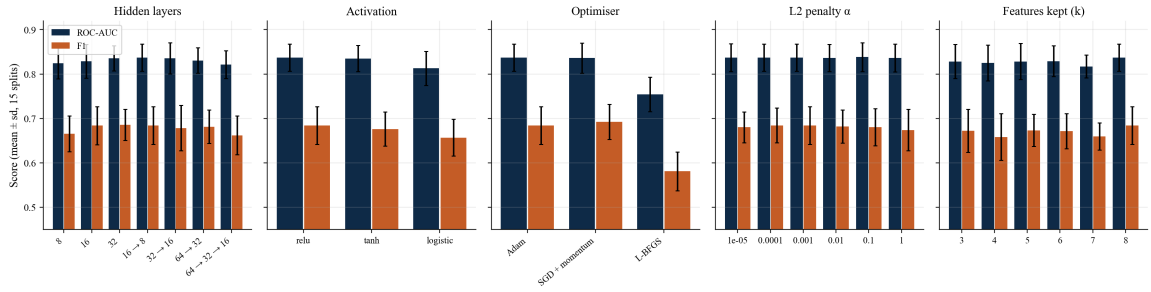


Figure 5.3: Architecture and hyperparameter ablation: ROC-AUC and F1 for each setting (mean ± sd over 15 splits).

Depth and width. A single hidden layer of 8 units is a bit weaker (ROC-AUC 0.824), and the deepest network (64–32–16) is weaker too (0.821) because it over-fits. Networks with 16 to 64 hidden units in one or two layers all sit within 0.01 of each other (0.829–0.837). The proposed 16–8 network is on that plateau; it was not cherry-picked. **Activation.** ReLU and tanh are equivalent (0.837 and 0.835). The logistic activation trains more slowly and does worse (0.813), which matches the vanishing-gradient argument for saturating units [45]. **Optimiser.** SGD with momentum matches Adam (0.836), but L-BFGS, which in scikit-learn does not use early stopping, over-fits badly (0.754). That is another reminder that early stopping is doing real work.

L2 penalty. ROC-AUC is flat (0.836–0.838) for α between 10^{-5} and 1. At α = 1 the network becomes more conservative and trades specificity for sensitivity (0.786). With early stopping already in place, L2 is a secondary regulariser here. **Number of features.** Every k < 8 gives a ROC-AUC at or below the all-feature result (0.837). The values for k = 3 to 7 (0.817–0.829) differ by less than one standard deviation, so no subset beats using all eight inputs. Across this grid the proposed design is robust: nothing in the range is better by more than the noise.

5.4 Leakage Ablation

To see how protocol shortcuts inflate the numbers, the proposed ANN and the random forest were run under four deliberately leaky protocols and compared with the leakage-safe protocol on the same splits (Table 5.3): (i) fitting the imputer and scaler on all 768 rows before CV; (ii) also selecting five features with SelectKBest on all rows; (iii) oversampling all rows before CV; and (iv) applying SMOTE to all rows before CV.

Table 5.3: Leakage ablation: leakage-safe protocol versus four leaky protocols (protocol P3)

Model	Protocol	F1	ROC-AUC	PR-AUC	Δ F1	Δ PR-AUC
ANN (MLP)	Leakage-safe (proposed)	0.684	0.837	0.706	—	—
	Impute+scale on all rows	0.683	0.840	0.715	-0.001	+0.009
	+SelectKBest on all rows	0.678	0.831	0.710	-0.006	+0.004
	Oversample before CV	0.771	0.850	0.825	+0.087	+0.119
	SMOTE before CV	0.780	0.856	0.838	+0.096	+0.132
Random forest	Leakage-safe (proposed)	0.696	0.836	0.711	—	—
	Impute+scale on all rows	0.694	0.836	0.710	-0.002	-0.001
	+SelectKBest on all rows	0.692	0.836	0.706	-0.004	-0.006
	Oversample before CV	0.815	0.881	0.861	+0.119	+0.150
	SMOTE before CV	0.816	0.882	0.865	+0.119	+0.153

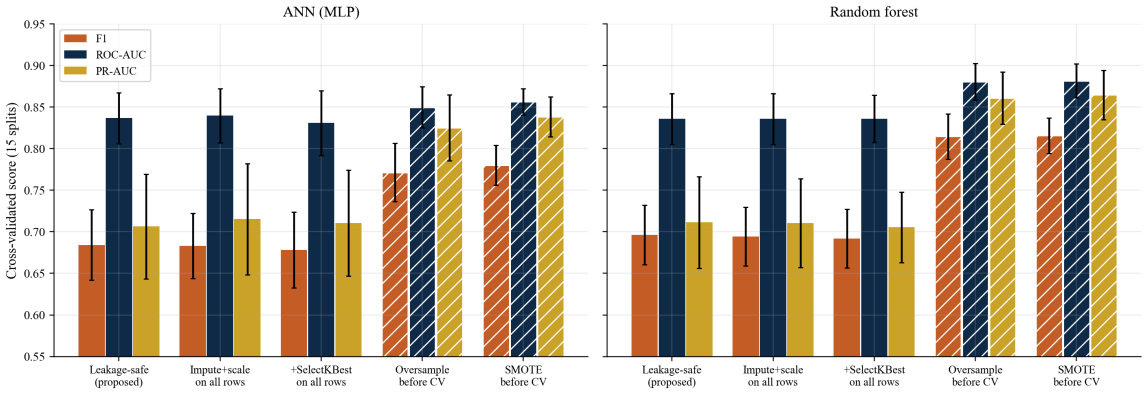


Figure 5.4: Effect of leaky protocols on F1, ROC-AUC, and PR-AUC for the ANN (left) and the random forest (right); hatched bars are leaky.

Fitting the imputer and scaler on the full table barely moves the scores ($|\Delta| \leq 0.01$). Medians, means, and variances of 768 rows are almost the same as those of 614 rows. Global feature selection also does little here, because the strongest features are obvious. Resampling before splitting is another story. Duplicated (or interpolated) minority patients then sit in both the training and test folds, and the models get credit for recognising them (Figure 5.4). For the ANN, oversampling before CV raises F1 by 0.087 and PR-AUC by 0.119. For the random forest, which can memorise duplicates in its leaves, SMOTE before CV raises F1 by 0.119, ROC-AUC by 0.046, and PR-AUC by 0.153. Those gains are artefacts of the protocol; nothing about the models changed. Resampling also balances the evaluation set, so the leaky accuracy and precision no longer describe the 65:35 population at all.

5.5 Nested versus Non-Nested Hyperparameter Selection

A 16-point grid over hidden layers {(8), (16), (16, 8), (32, 16)} and α {10^{-4}, …, 10^{-1}} was searched in two ways. The non-nested way scores every grid point with the same 5-fold CV and quotes the best one: (16) with α = 0.1, ROC-AUC 0.835. The nested way picks the grid point with an inner 5-fold CV on each outer training fold and scores it on the outer test fold (Table 5.4). The nested estimate is 0.819 ± 0.028, 0.016 lower, and the chosen setup changes from fold to fold. That is the selection bias described by Varma and Simon [33] and Cawley and Talbot [34], measured on this table. It is also why the proposed configuration was fixed in advance rather than picked from the results. The console output of the leakage and nested experiments is in Figure 5.5.

Table 5.4: Nested cross-validation: hyperparameters selected in each outer fold and the resulting outer-fold ROC-AUC

Outer fold	Selected hidden layers	Selected α	Inner CV ROC-AUC	Outer-fold ROC-AUC
1	(16, 8)	0.1	0.834	0.830
2	(32, 16)	0.1	0.839	0.849
3	(32, 16)	0.001	0.846	0.806
4	(32, 16)	0.1	0.843	0.832
5	(16,)	0.001	0.853	0.777

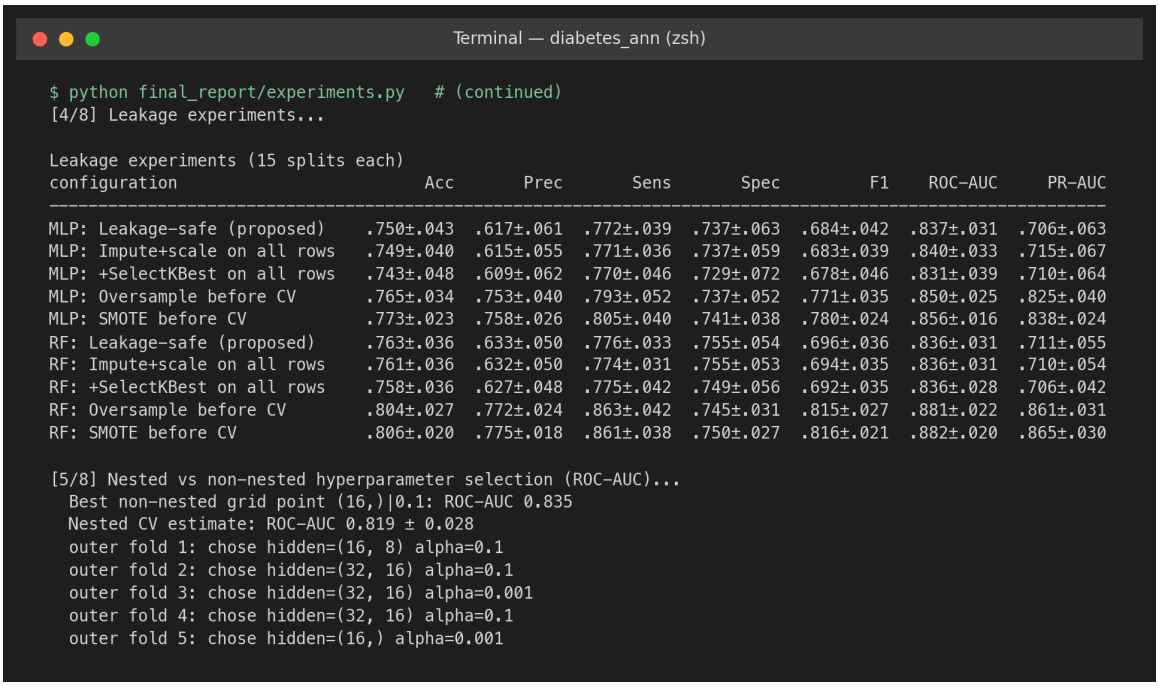


Figure 5.5: Console output of the leakage experiments and the nested versus non-nested comparison.

5.6 Seed Stability

Figure 5.6 repeats the proposed system's 5-fold CV with ten other seeds (0–9) for the network initialisation and the oversampler, keeping the folds fixed. ROC-AUC ranges from 0.799 to 0.840 (mean 0.822 ± 0.013) and sensitivity from 0.679 to 0.769. The default seed 42 gives ROC-AUC 0.834, which is higher than eight of the ten other seeds, so the seed-42 figures in this report sit about 0.01 above the seed-averaged expectation. The seed-to-seed spread (≈ 0.04 in ROC-AUC) is as large as most of the ablation differences. Only the large effects in Section 5.2 should be treated as real.

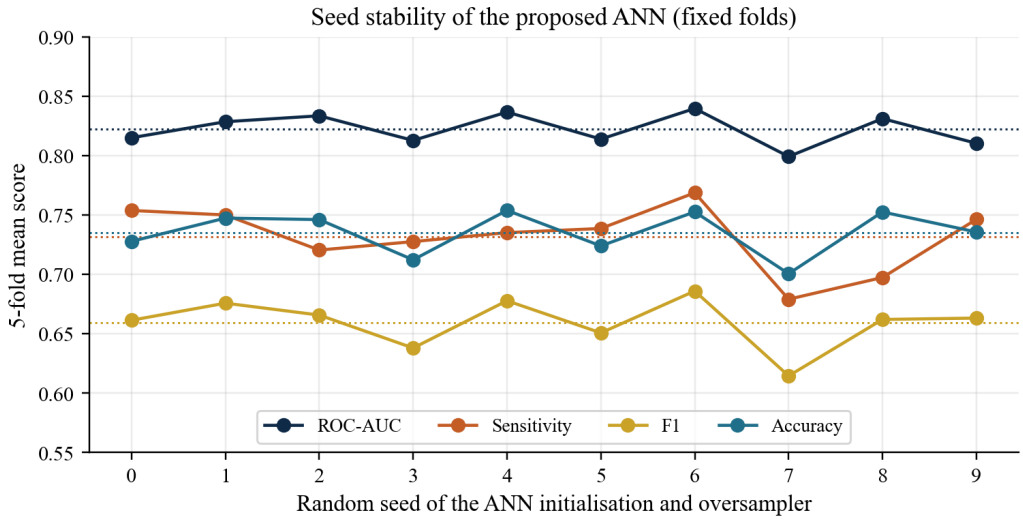


Figure 5.6: Seed stability of the proposed ANN: 5-fold mean scores for ten random seeds (dotted lines = seed means).

5.7 State-of-the-Art Comparison

Table 5.5 puts the proposed system next to published results on the same dataset. Accuracy is shown because it is the only metric most studies report. Sensitivity, specificity, and AUC are filled in where the authors quoted them. The evaluation-protocol column is the one that actually lets the numbers be compared.

Table 5.5: State-of-the-art comparison on the Pima Indians Diabetes dataset (published figures as reported by the authors)

Study	Method	Protocol	Acc. (%)	Sens.	Spec.	AUC
Smith et al. [10]	ADAP neural model	576/192 split	—	0.76	0.76	ROC shown
Kayaer & Yıldırım [20]	GRNN	Train/test split	80.21	—	—	—
Polat & Güneş [21]	PCA + ANFIS	10-fold CV	89.47	—	—	—
Kahramanli & Allahverdi [22]	Hybrid ANN + FNN	k-fold CV	84.24	—	—	—
Temurtas et al. [23]	MLNN (LM)	Split / 10-fold CV	82.37	—	—	—
Erkaymaz et al. [24]	Small-world FFANN	k-fold CV	93.06	0.85	0.96	—
Maniruzzaman et al. [25]	Gaussian process	K-fold CV	81.97	0.918	0.633	—
Sisodia & Sisodia [26]	Naïve Bayes	Cross-validation	76.30	—	—	—
Hasan et al. [30]	Weighted ensemble	k-fold CV	—	0.789	0.934	0.950
Joshi & Dhakal [28]	Logistic regression	Split + CV	78.26	—	—	—
Kumari et al. [31]	Soft-voting ensemble	Train/test	79.04	—	—	—
Howlader et al. [41]	Gen. boosted regression	Cross-validation	90.91	—	—	—
Chang et al. [29]	Random forest	70/30 split	79.57	—	—	0.862
This work (proposed)	**Leakage-safe ANN + ROS**	**10×5 repeated CV**	**74.57**	**0.768**	**0.734**	**0.834**
This work (proposed)	**Leakage-safe ANN + ROS**	**80/20 hold-out**	**74.68**	**0.796**	**0.720**	**0.821**
This work (leaky, for reference)	Random forest, SMOTE before CV	3×5 repeated CV	80.57	0.861	0.750	0.882

The table falls into three groups. **(a) Studies with simple models and simple protocols** (logistic regression, naïve Bayes, random forests, soft voting, GRNN) quote 76–82% accuracy, the same range as this work's 74.6% (repeated CV) and 74.7% (hold-out). The proposed ANN therefore sits with the honest state of the art on this benchmark. Its sensitivity–specificity pair is close to the 0.76/0.76 of Smith et al. and more balanced than the 0.918/0.633 of the Gaussian-process model. **(b) Studies quoting 84–93% accuracy or AUC ≈ 0.95** use heavier pipelines (PCA, fuzzy hybrids, rewired topologies, outlier removal, ensembles). On a 768-row table those numbers depend a lot on exactly when preprocessing, selection, and tuning are fitted. **(c) This work's leaky reference row** shows that a standard random forest reaches ROC-AUC 0.882 and sensitivity 0.861 just by applying SMOTE before cross-validation.

We are not accusing any one paper of leakage. We are saying protocol choices alone can move the numbers by about as much as the gap between group (a) and group (b). If a Pima write-up does not say how preprocessing, resampling and tuning sat inside the validation, accuracy is not a fair comparison. Against the papers that used ordinary, transparent protocols, our leakage-safe ANN sits in the same band. It also reports the stuff most of them skip: full metrics, tests against baselines, an ablation, and a direct estimate of how much the usual shortcuts inflate the score.

5.8 Summary of Findings

- The proposed ANN reaches ROC-AUC 0.834, sensitivity 0.768, and specificity 0.734 under repeated CV, and similar values on the hold-out set.
- In ranking quality it is statistically indistinguishable from logistic regression and a random forest. What it does offer is a screening-oriented operating point, created by training-fold oversampling.
- Standardisation, early stopping, and zero recoding are the pieces that matter. Architecture, activation (except logistic), L2 strength, and feature selection do not.
- Resampling before splitting inflates PR-AUC by 0.12–0.15 and F1 by 0.09–0.12. Non-nested tuning inflates ROC-AUC by about 0.02.
- Seed-to-seed variation (≈ 0.04 in ROC-AUC) is as large as most configuration differences, so only the large effects should be trusted.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

We set out to put an ANN at the centre of a Diabetes Prediction System and to score it without cheating on the Pima table. The table is small. Classes are skewed. Zeros sit in columns where a living adult cannot have a zero. Published accuracies still run from the mid-70s to well over 90%, and that spread is what made us suspicious in the first place. So training a net was only half the job. The other half was asking what the net actually does when every learned step stays off the evaluation rows. If the honest score landed near logistic regression, that was still a result worth writing down.

Impossible zeros in five columns are recoded. Valid zero pregnancies are left alone. Training-fold medians fill holes. Inputs are scaled. The minority class is oversampled only inside the training fold. Then an 8–16–8–1 MLP is trained with ReLU, a sigmoid, Adam, L2 and early stopping. We wrote the method out (equations and algorithms), put it in a tested Python package, and scored it with seeded CV, repeated CV, one hold-out, corrected tests and bootstrap intervals. None of that is fancy. It is just the stuff that has to sit inside the fold if you want the number to mean anything.

On 10 × 5 repeated CV we got ROC-AUC 0.834, sensitivity 0.768, specificity 0.734. Hold-out found 43 of 54 diabetic patients. Ranking was a dead heat with logistic regression and the random forest. That is not a failure. Eight mostly monotonic predictors do not leave a lot of extra signal for a neural net to grab. The practical difference was the operating point. Training-fold oversampling raised sensitivity by about 0.21 and left ranking alone, which is the cut you want if missed cases are costly. Permutation importance put glucose first, then age, BMI, pregnancies. That matches the usual type 2 story. No surprise there.

Ablation sorted the priorities. Scaling, early stopping and the medical zero rule moved the scores. Depth, width, ReLU versus tanh, L2 strength and univariate selection sat inside seed noise. Oversample before CV and PR-AUC inflates 0.12 to 0.15; F1 by up to 0.12. Pick hyperparameters on the evaluation folds and you gift yourself about 0.02 ROC-AUC. Ten seeds already swing ROC-AUC by about 0.04. So we stopped trusting small gaps. Those measured bumps also explain why the published Pima numbers sit so far apart.

Limits are not subtle. One community of adult women, exams from decades ago, a historical glucose rule, nearly half of insulin imputed, no external cohort. Educational risk estimate. Not a diagnostic device. Inside those limits a small, regularised ANN is a decent Pima classifier. The protocol deserves as much attention as which network you pick. We would not ship this to a clinic. We would use it to teach the pipeline: recode the zeros properly, keep the test fold clean, and stop quoting accuracy as if it told the whole story. That is the actual takeaway from the semester.

6.2 Future Work

- Run the same pipeline on an external, mixed-sex, contemporary cohort collected with consent and ethics approval. Report calibration, not only ranking.

- Swap the single median fill for multiple imputation so the 48.7% missing insulin values show up as uncertainty in the predictions, not as a fake certainty.
- Set the decision threshold from actual costs of missed cases versus false alarms, on a validation set or with nested CV. Not from the evaluation predictions.
- Put a local explanation next to each call (SHAP would do) and check whether the net actually learns interactions a linear model cannot see.
- Wrap the trained pipeline as a small web or mobile demo with input checks and a clear non-diagnostic disclaimer. Lab use only.

REFERENCES

[1] World Health Organization, "Diabetes," Fact sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/diabetes (accessed Oct. 2026).

[2] World Health Organization, "Urgent action needed as global diabetes cases increase four-fold over past decades," News release, Nov. 13, 2024. [Online]. Available: https://www.who.int/news/item/13-11-2024-urgent-action-needed-as-global-diabetes-cases-increase-four-fold-over-past-decades

[3] H. Sun et al., "IDF Diabetes Atlas: Global, regional and country-level diabetes prevalence estimates for 2021 and projections for 2045," *Diabetes Res. Clin. Pract.*, vol. 183, Art. no. 109119, 2022, doi: 10.1016/j.diabres.2021.109119.

[4] American Diabetes Association Professional Practice Committee, "2. Diagnosis and classification of diabetes: Standards of Care in Diabetes—2024," *Diabetes Care*, vol. 47, suppl. 1, pp. S20–S42, 2024, doi: 10.2337/dc24-S002.

[5] I. Kavakiotis, O. Tsave, A. Salifoglou, N. Maglaveras, I. Vlahavas, and I. Chouvarda, "Machine learning and data mining methods in diabetes research," *Comput. Struct. Biotechnol. J.*, vol. 15, pp. 104–116, 2017, doi: 10.1016/j.csbj.2016.12.005.

[6] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989, doi: 10.1016/0893-6080(89)90020-8.

[7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986, doi: 10.1038/323533a0.

[8] P. H. Bennett, T. A. Burch, and M. Miller, "Diabetes mellitus in American (Pima) Indians," *Lancet*, vol. 2, no. 7716, pp. 125–128, 1971, doi: 10.1016/S0140-6736(71)92303-8.

[9] W. C. Knowler, P. H. Bennett, R. F. Hamman, and M. Miller, "Diabetes incidence and prevalence in Pima Indians: A 19-fold greater incidence than in Rochester, Minnesota," *Am. J. Epidemiol.*, vol. 108, no. 6, pp. 497–505, 1978, doi: 10.1093/oxfordjournals.aje.a112648.

[10] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proc. Annu. Symp. Comput. Appl. Med. Care*, 1988, pp. 261–265.

[11] V. Sigillito (donor), "Pima Indians Diabetes Database," National Institute of Diabetes and Digestive and Kidney Diseases (original owner); donated to the UCI Machine Learning Repository, May 9, 1990.

[12] F. Leisch and E. Dimitriadou, "mlbench: Machine learning benchmark problems," R package, help page "PimaIndiansDiabetes." [Online]. Available: https://search.r-project.org/CRAN/refmans/mlbench/html/PimaIndiansDiabetes.html

[13] B. D. Ripley, *Pattern Recognition and Neural Networks*. Cambridge, U.K.: Cambridge Univ. Press, 1996.

[14] G. Wahba, C. Gu, Y. Wang, and R. Chappell, "Soft classification, a.k.a. risk estimation, via penalized log likelihood and smoothing spline analysis of variance," in *The Mathematics of Generalization*, D. H. Wolpert, Ed. Reading, MA, USA: Addison-Wesley, 1995, pp. 331–359.

[15] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, Art. no. 15, 2012, doi: 10.1145/2382577.2382579.

[16] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[17] M. R. Bozkurt, N. Yurtay, Z. Yılmaz, and C. Sertkaya, "Comparison of different methods for determining diabetes," *Turk. J. Elect. Eng. Comput. Sci.*, vol. 22, no. 4, pp. 1044–1055, 2014, doi: 10.3906/elk-1209-82.

[18] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[19] G. S. Collins, J. B. Reitsma, D. G. Altman, and K. G. M. Moons, "Transparent reporting of a multivariable prediction model for individual prognosis or diagnosis (TRIPOD): The TRIPOD statement," *BMJ*, vol. 350, Art. no. g7594, 2015, doi: 10.1136/bmj.g7594.

[20] K. Kayaer and T. Yıldırım, "Medical diagnosis on Pima Indian diabetes using general regression neural networks," in *Proc. Int. Conf. Artif. Neural Netw. Neural Inf. Process. (ICANN/ICONIP)*, Istanbul, Turkey, 2003, pp. 181–184.

[21] K. Polat and S. Güneş, "An expert system approach based on principal component analysis and adaptive neuro-fuzzy inference system to diagnosis of diabetes disease," *Digit. Signal Process.*, vol. 17, no. 4, pp. 702–710, 2007, doi: 10.1016/j.dsp.2006.09.005.

[22] H. Kahramanli and N. Allahverdi, "Design of a hybrid system for the diabetes and heart diseases," *Expert Syst. Appl.*, vol. 35, no. 1–2, pp. 82–89, 2008, doi: 10.1016/j.eswa.2007.06.004.

[23] H. Temurtas, N. Yumusak, and F. Temurtas, "A comparative study on diabetes disease diagnosis using neural networks," *Expert Syst. Appl.*, vol. 36, no. 4, pp. 8610–8615, 2009, doi: 10.1016/j.eswa.2008.10.032.

[24] O. Erkaymaz, M. Ozer, and M. Perc, "Performance of small-world feedforward neural networks for the diagnosis of diabetes," *Appl. Math. Comput.*, vol. 311, pp. 22–28, 2017, doi: 10.1016/j.amc.2017.05.010.

[25] M. Maniruzzaman, N. Kumar, M. M. Abedin, M. S. Islam, H. S. Suri, A. S. El-Baz, and J. S. Suri, "Comparative approaches for classification of diabetes mellitus data: Machine learning paradigm," *Comput. Methods Programs Biomed.*, vol. 152, pp. 23–34, 2017, doi: 10.1016/j.cmpb.2017.09.004.

[26] D. Sisodia and D. S. Sisodia, "Prediction of diabetes using classification algorithms," *Procedia Comput. Sci.*, vol. 132, pp. 1578–1585, 2018, doi: 10.1016/j.procs.2018.05.122.

[27] Q. Zou, K. Qu, Y. Luo, D. Yin, Y. Ju, and H. Tang, "Predicting diabetes mellitus with machine learning techniques," *Front. Genet.*, vol. 9, Art. no. 515, 2018, doi: 10.3389/fgene.2018.00515.

[28] R. D. Joshi and C. K. Dhakal, "Predicting type 2 diabetes using logistic regression and machine learning approaches," *Int. J. Environ. Res. Public Health*, vol. 18, no. 14, Art. no. 7346, 2021, doi: 10.3390/ijerph18147346.

[29] V. Chang, J. Bailey, Q. A. Xu, and Z. Sun, "Pima Indians diabetes mellitus classification based on machine learning (ML) algorithms," *Neural Comput. Appl.*, vol. 35, pp. 16157–16173, 2023, doi: 10.1007/s00521-022-07049-z.

[30] M. K. Hasan, M. A. Alam, D. Das, E. Hossain, and M. Hasan, "Diabetes prediction using ensembling of different machine learning classifiers," *IEEE Access*, vol. 8, pp. 76516–76531, 2020, doi: 10.1109/ACCESS.2020.2989857.

[31] S. Kumari, D. Kumar, and M. Mittal, "An ensemble approach for classification and prediction of diabetes mellitus using soft voting classifier," *Int. J. Cogn. Comput. Eng.*, vol. 2, pp. 40–46, 2021, doi: 10.1016/j.ijcce.2021.01.001.

[32] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artif. Intell. (IJCAI)*, 1995, pp. 1137–1145.

[33] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, Art. no. 91, 2006, doi: 10.1186/1471-2105-7-91.

[34] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 2079–2107, 2010.

[35] scikit-learn developers, "Common pitfalls and recommended practices," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html

[36] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.

[37] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[38] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[39] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240, doi: 10.1145/1143844.1143874.

[40] R. J. A. Little and D. B. Rubin, *Statistical Analysis With Missing Data*, 3rd ed. Hoboken, NJ, USA: Wiley, 2019.

[41] K. C. Howlader, M. S. Satu, M. A. Awal, M. R. Islam, S. M. S. Islam, J. M. W. Quinn, and M. A. Moni, "Machine learning models for classification and

identification of significant attributes to detect type 2 diabetes," *Health Inf. Sci. Syst.*, vol. 10, no. 1, Art. no. 2, 2022, doi: 10.1007/s13755-021-00168-2.

[42] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.

[43] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.

[44] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.

[45] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2010, pp. 249–256.

[46] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, 2015, arXiv:1412.6980.

[48] L. Prechelt, "Early stopping—But when?," in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 55–69, doi: 10.1007/3-540-49430-8_3.

[49] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Statist. Soc. B*, vol. 20, no. 2, pp. 215–242, 1958.

[50] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[51] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982, doi: 10.1148/radiology.143.1.7063747.

[52] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950, doi: 10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3.

[53] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Mach. Learn.*, vol. 52, no. 3, pp. 239–281, 2003, doi: 10.1023/A:1024068626366.

[54] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[55] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[56] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[57] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf. (SciPy)*, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[58] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.